

Table of Contents

1. Easy Tasks (Task 1 – Task 10)
2. Medium Tasks (Task 11 – Task 25)
3. Hard Tasks (Task 26 – Task 50)
4. Topic Coverage Matrix

1. Easy Tasks

Task 1 – Task 10 | Fundamental concepts and straightforward implementations.

Task 1 — Personal Profile / Bio Page

Difficulty: Easy

Topics Covered:

- HTML Structure & Semantic Elements
- HTML Text, Headings & Lists
- CSS Colors & Backgrounds
- CSS Text, Fonts & Units

Objective: Build a single-page personal profile using correct HTML structure and simple CSS styling.

Task Description: A junior developer has been asked to create a personal 'About Me' page for a company directory. The page should present the candidate's photo placeholder, name, short bio, and contact details using clean, semantic markup.

Functional Requirements:

- Display a profile picture placeholder, full name, and job title
- Include a short biography paragraph (3-4 sentences)
- List contact details (email, phone, location) using an unordered list
- Use proper heading hierarchy (h1, h2, h3) for sections

Technical Requirements: Use semantic HTML elements (header, main, section, footer). Apply CSS for colors, background, font choice, text alignment, and spacing.

Expected Output: A single static HTML page with a clearly structured profile layout, readable typography, and consistent color scheme.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Semantic tags are used instead of generic divs where appropriate
- ☐ Heading hierarchy is logical and consistent
- ☐ Contact list is present and correctly marked up
- ☐ Page is styled with readable colors and fonts
- ☐ No major functional or markup issues

Task 2 — Product Info Card using the CSS Box Model

Difficulty: Easy

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Box Model (Margin/Padding/Border/Box-sizing)
- CSS Colors & Backgrounds
- CSS Selectors & Combinators

Objective: Design a single product card that correctly demonstrates the CSS box model.

Task Description: An online store needs a reusable product card component showing an image placeholder, product name, description, and price, with clearly visible spacing and borders.

Functional Requirements:

- Card contains an image placeholder, title, short description, and price
- Card has visible padding between content and border

- Card has a margin separating it from the page edge
- Border and rounded corners are applied to the card

Technical Requirements: Use box-sizing, padding, margin, and border properties. Apply class and element selectors to style the card.

Expected Output: A single, well-spaced product card centered on the page with clear visual separation between content and edges.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Box model properties (margin, padding, border) are all visibly applied
- ☐ box-sizing is used correctly so sizes render as expected
- ☐ Card content does not touch the card border
- ☐ Layout matches the requirements
- ☐ No major functional issues

Task 3 — Basic Registration Form

Difficulty: Easy

Topics Covered:

- HTML Forms & Input Elements
- CSS Styling of Lists/Tables/Forms/Images
- CSS Box Model (Margin/Padding/Border/Box-sizing)

Objective: Create a simple, styled registration form using core HTML form elements.

Task Description: A community website needs a basic sign-up form for new members to register with their personal details.

Functional Requirements:

- Collect full name, email, password, and date of birth
- Include a gender selection using radio buttons

- Include a 'Submit' and a 'Reset' button
- Use label elements correctly associated with each input

Technical Requirements: Use form, input, label, and button elements with appropriate type and required attributes. Style inputs and buttons using CSS.

Expected Output: A vertically stacked registration form with labeled fields, visible input borders, and styled buttons.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The image shows a structural reference mockup of a 'Create Account' form. The form is titled 'Create Account' with a blue icon. It contains four input fields: 'FULL NAME', 'EMAIL', 'PASSWORD', and 'DATE OF BIRTH'. Each field has a light blue border and a light blue background. Below the fields is a blue 'REGISTER' button.

Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ All required fields are present with correct input types
- ☐ Labels are correctly associated with their inputs
- ☐ Submit and Reset buttons function as expected
- ☐ Form is legible and consistently styled
- ☐ No major functional issues

Task 4 — Simple Horizontal Navigation Bar

Difficulty: Easy

Topics Covered:

- HTML Links, Navigation & Media
- CSS Flexbox
- CSS Pseudo-classes & Pseudo-elements
- CSS Selectors & Combinators

Objective: Build a horizontal navigation bar with hover feedback using Flexbox.

Task Description: A small business site needs a clean top navigation bar with a logo on the left and menu links on the right that respond visually when hovered.

Functional Requirements:

- Display a logo/brand name on the left
- Display at least 4 navigation links on the right
- Links change color or background on hover
- Bar remains a single row on standard desktop width

Technical Requirements: Use an HTML nav element containing an unordered list of links. Use Flexbox for alignment and the :hover pseudo-class for interaction.

Expected Output: A single-row navigation bar with evenly spaced links and a visible hover effect.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Navigation bar uses nav and list elements correctly
- ☐ Flexbox is used to align logo and links
- ☐ Hover state is clearly visible on all links
- ☐ Layout matches requirements
- ☐ No major functional issues

Task 5 — Basic Landing Page Hero Banner

Difficulty: Easy

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Colors & Backgrounds
- CSS Text, Fonts & Units
- CSS Display & Positioning

Objective: Create a single hero banner section for a landing page with a heading, subtext, and call-to-action button.

Task Description: A startup wants a homepage hero section that immediately communicates its value proposition to visitors.

Functional Requirements:

- Display a large heading and a supporting subheading
- Include a call-to-action button

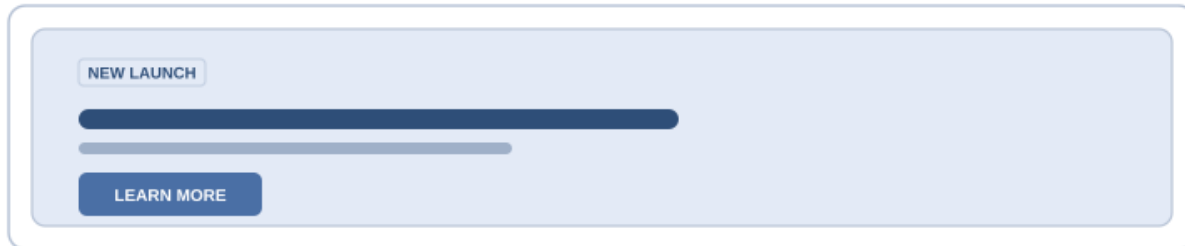
- Center all content horizontally and vertically within the banner
- Apply a distinct background color to the section

Technical Requirements: Use CSS for background-color, text-align, and centering techniques covered in the syllabus (flexbox or position-based centering).

Expected Output: A full-width banner section with centered heading, subtext, and a styled button.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Heading, subtext, and button are all present
- ☐ Content is properly centered within the banner
- ☐ Background color is applied and visually distinct
- ☐ Button is clearly styled and clickable-looking
- ☐ No major functional issues

Task 6 — Styled Navigation List Menu

Difficulty: Easy

Topics Covered:

- HTML Text, Headings & Lists
- CSS Styling of Lists/Tables/Forms/Images
- CSS Pseudo-classes & Pseudo-elements

Objective: Style an HTML list into a vertical menu with custom markers and hover states.

Task Description: A documentation site needs a simple sidebar-style menu built from a plain HTML list, styled to look like a clickable navigation menu.

Functional Requirements:

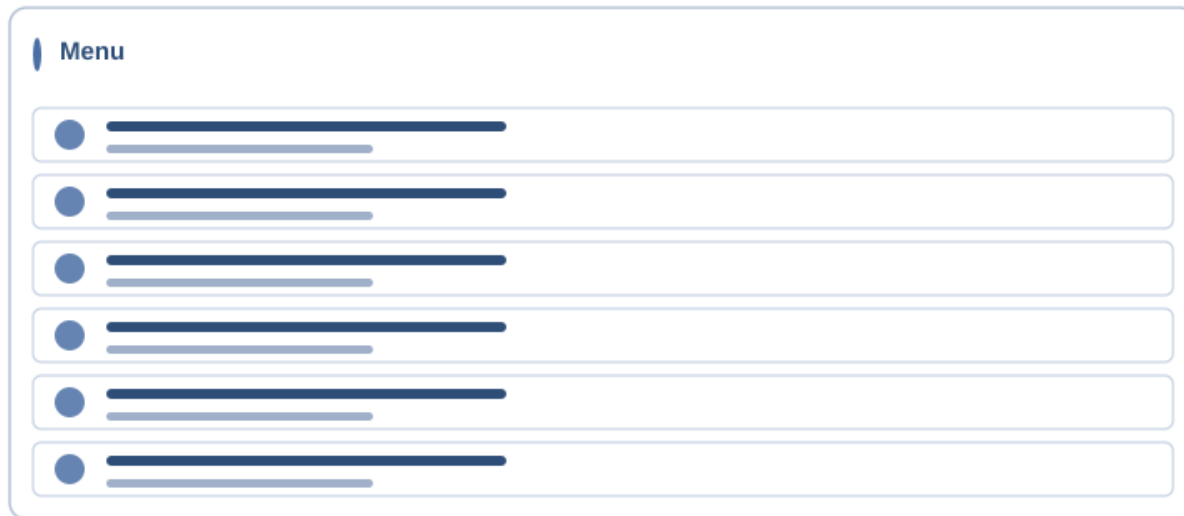
- Create a vertical list of at least 6 menu items
- Remove default list bullet markers
- Apply padding so each item looks like a clickable row
- Highlight an item when hovered

Technical Requirements: Use an unordered list with list-style removed, custom padding/background via CSS, and the :hover pseudo-class.

Expected Output: A clean vertical menu list with consistent spacing and a visible hover highlight on each row.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Default bullets are removed from the list
- ☐ Each menu item has consistent padding and spacing
- ☐ Hover effect is visible and consistent across items
- ☐ Markup uses list elements correctly
- ☐ No major functional issues

Task 7 — Simple Responsive Image Gallery

Difficulty: Easy

Topics Covered:

- HTML Links, Navigation & Media
- CSS Grid
- CSS Responsive Design & Media Queries
- CSS Math Functions (calc/min/max/clamp)

Objective: Build a simple grid-based image gallery that reflows on smaller screens.

Task Description: A photography hobbyist wants a basic gallery page to showcase eight photos in a neat grid that adapts to different screen widths.

Functional Requirements:

- Display 8 image placeholders in a grid layout
- Grid shows 4 columns on desktop and fewer columns on narrow screens
- Maintain consistent spacing (gap) between images

- Images/placeholders maintain a consistent aspect ratio

Technical Requirements: Use CSS Grid (grid-template-columns, gap), the minmax() math function to size columns flexibly, and at least one media query to adjust column count responsively.

Expected Output: A responsive photo grid that rearranges from 4 columns to 2 or 1 column as the viewport narrows.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output at two screen widths

Acceptance Criteria:

- ☐ Grid displays exactly 8 items with consistent gaps
- ☐ Layout is built using CSS Grid, not floats or tables
- ☐ At least one media query changes the column count
- ☐ Gallery remains visually balanced at different widths
- ☐ No major functional issues

Task 8 — Basic Pricing / Data Table

Difficulty: Easy

Topics Covered:

- HTML Tables
- CSS Styling of Lists/Tables/Forms/Images
- CSS Pseudo-classes & Pseudo-elements

Objective: Create a styled HTML table comparing plan features, including zebra striping and hover effects.

Task Description: A SaaS product needs a simple plan comparison table on its pricing page, listing features across three plan tiers.

Functional Requirements:

- Table has a header row identifying each plan (Basic, Standard, Premium)

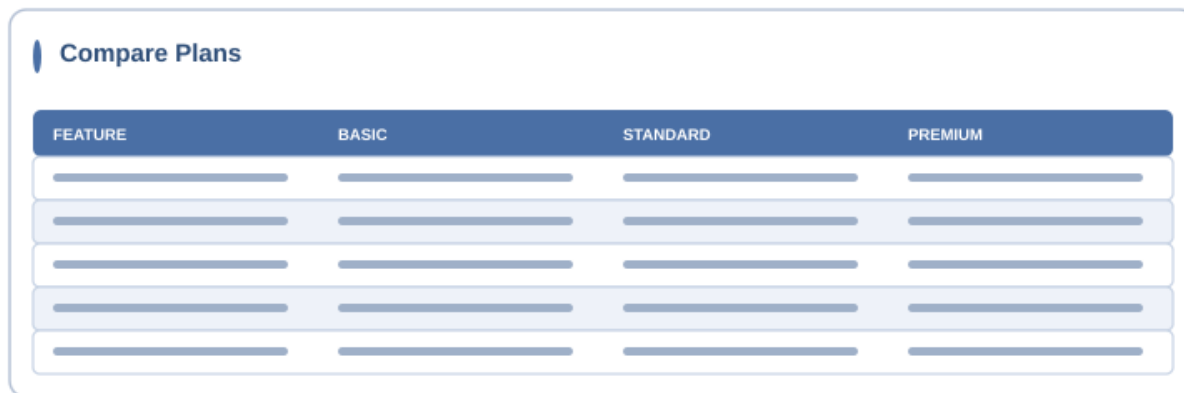
- At least 4 feature rows compare values across plans
- Alternating row colors (zebra striping) are applied
- Rows highlight when hovered

Technical Requirements: Use table, thead, tbody, tr, th, and td elements. Style using border-collapse, cell padding, nth-child, and :hover.

Expected Output: A clean comparison table with a styled header, striped rows, and a hover highlight effect.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



The mockup shows a table titled 'Compare Plans' with four columns: FEATURE, BASIC, STANDARD, and PREMIUM. There are five data rows. The first row is highlighted with a light blue background. The second row has a light blue background. The third row is white. The fourth row has a light blue background. The fifth row is white. The table has a dark blue header and alternating row colors (zebra striping). The table is enclosed in a light blue border.

FEATURE	BASIC	STANDARD	PREMIUM

Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Table uses correct semantic table elements
- ☐ Zebra striping is applied using CSS, not manual classes per row
- ☐ Hover effect is visible on data rows
- ☐ Table is legible with adequate cell padding
- ☐ No major functional issues

Task 9 — Typography & Text Styling Showcase Page

Difficulty: Easy

Topics Covered:

- HTML Text, Headings & Lists
- CSS Text, Fonts & Units
- CSS Selectors & Combinators

Objective: Build a reference page demonstrating a range of CSS text formatting properties.

Task Description: A design team wants a single reference page that demonstrates the site's typography rules for headings, paragraphs, and emphasized text.

Functional Requirements:

- Demonstrate at least 3 heading levels with distinct sizes

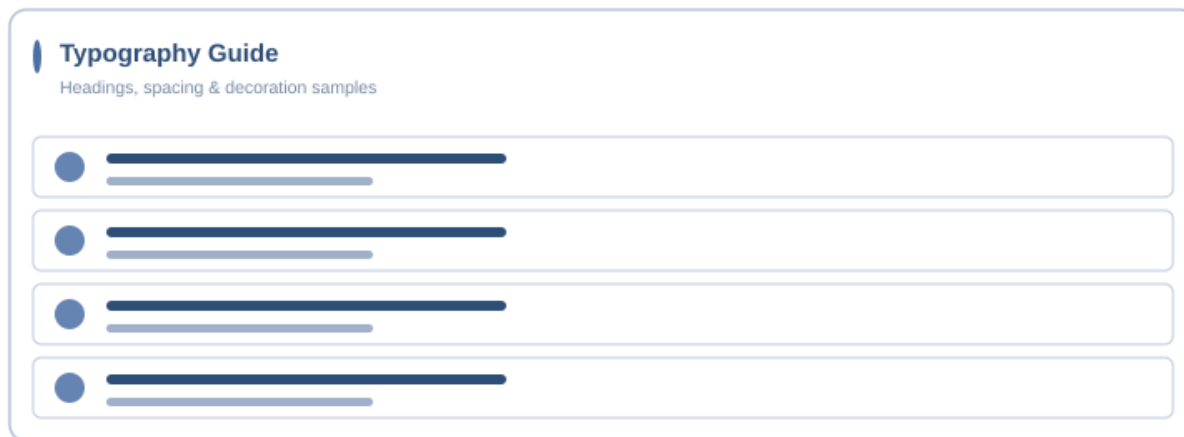
- Demonstrate text alignment, letter spacing, and line height on a paragraph
- Demonstrate text decoration and text transformation on sample text
- Use at least two different CSS units (e.g., px, em, rem, %)

Technical Requirements: Use CSS text formatting properties including text-align, letter-spacing, line-height, text-decoration, text-transform, and absolute/relative units.

Expected Output: A single page with clearly labeled examples of each typography rule applied to sample text.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ At least 3 heading levels are demonstrated with visible size differences
- ☐ Text alignment, spacing, and line-height examples are present
- ☐ Text decoration and transformation examples are present
- ☐ At least two distinct CSS units are used
- ☐ No major functional issues

Task 10 — Interactive Click Counter

Difficulty: Easy

Topics Covered:

- HTML Structure & Semantic Elements
- JS Variables, Data Types & Scope
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a simple counter widget that increments a displayed number when a button is clicked.

Task Description: A simple engagement widget is needed for a demo page: a button that visitors can click, with a live count of total clicks displayed on screen.

Functional Requirements:

- Display a numeric counter starting at 0
- Provide an 'Increase' button that increments the counter by 1 on click
- Provide a 'Reset' button that sets the counter back to 0
- Update the displayed number immediately using the DOM (no page reload)

Technical Requirements: Use a JavaScript variable to store the count, addEventListener for click events, and DOM methods (getElementById/querySelector + innerHTML or textContent) to update the page.

Expected Output: A counter widget where clicking 'Increase' visibly updates the number and 'Reset' returns it to zero, without reloading the page.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ Counter starts at 0 on page load
- ☐ Increase button correctly increments the display
- ☐ Reset button correctly resets the display to 0
- ☐ DOM is updated without a page reload
- ☐ No major functional issues

2. Medium Tasks

Task 11 – Task 25 | Multiple concepts combined with interactive or responsive behavior.

Task 11 — Responsive Personal Portfolio Landing Page

Difficulty: Medium

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Flexbox
- CSS Grid
- CSS Responsive Design & Media Queries
- CSS Pseudo-classes & Pseudo-elements

Objective: Build a multi-section, responsive one-page portfolio site for a freelance developer.

Task Description: A freelancer wants a single-page portfolio site with an intro/hero section, a skills section, a projects grid, and a contact section, all adapting cleanly to mobile screens.

Functional Requirements:

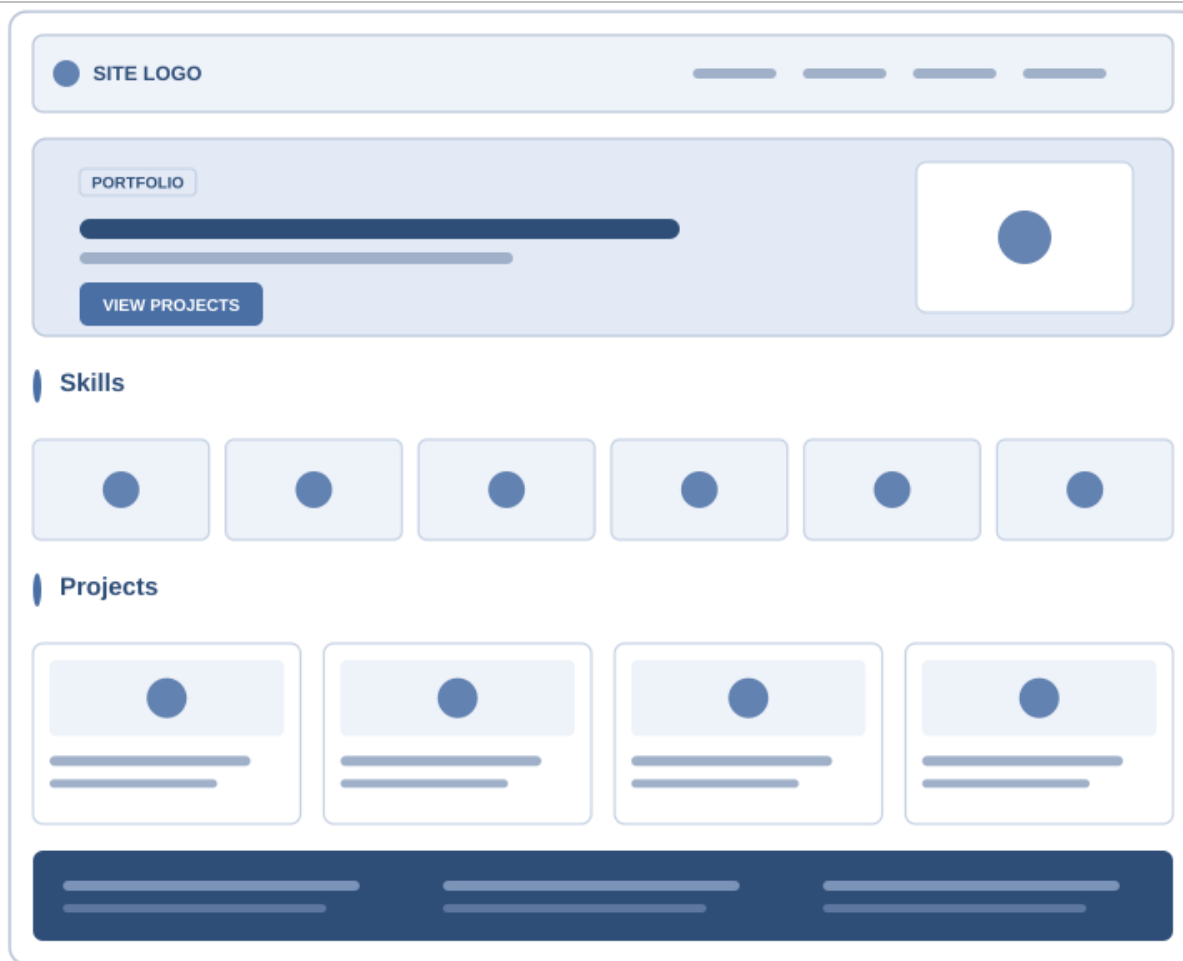
- Sticky or top navigation bar linking to each section via anchor links
- Hero section introducing the person with a call-to-action
- Skills section listing at least 6 skills in a flexible layout
- Projects section showing at least 4 project cards in a responsive grid
- Footer with contact/social links

Technical Requirements: Combine Flexbox and CSS Grid for different sections. Include at least two media query breakpoints. Use hover pseudo-classes on interactive elements.

Expected Output: A fully scrollable single-page portfolio that reflows from a multi-column desktop layout to a single-column mobile layout.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshots at desktop and mobile widths

Acceptance Criteria:

- ☐ All five sections (nav, hero, skills, projects, footer) are present
- ☐ Navigation links scroll to the correct sections
- ☐ Layout is responsive across at least two breakpoints
- ☐ Both Flexbox and Grid are used appropriately
- ☐ Hover states are present on interactive elements
- ☐ No major functional issues

Task 12 — Restaurant Menu Section with Categorized Dishes

Difficulty: **Medium**

Topics Covered:

- HTML Tables
- HTML Text, Headings & Lists
- CSS Grid

- CSS Styling of Lists/Tables/Forms/Images
- CSS Text, Fonts & Units

Objective: Build a restaurant menu page presenting dishes grouped by category with prices.

Task Description: A restaurant wants a digital menu page grouping dishes into Starters, Main Course, and Desserts, each showing dish name, short description, and price.

Functional Requirements:

- Group dishes under at least 3 category headings
- Each category displays at least 3 dishes with name, description, and price
- Use a consistent grid layout for dish listings within each category
- Visually distinguish category headings from dish entries

Technical Requirements: Use semantic headings for categories, CSS Grid or Flexbox for dish layout, and consistent typography rules for names/descriptions/prices.

Expected Output: A categorized, easy-to-scan menu page with clear visual grouping and consistent price formatting.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ At least 3 categories with at least 3 dishes each are present
 - ☐ Layout uses CSS Grid or Flexbox consistently
 - ☐ Prices are visually aligned and easy to scan
 - ☐ Category headings are visually distinct from dish content
 - ☐ No major functional issues
-

Task 13 — Responsive Three-Tier Pricing Plan Cards

Difficulty: Medium

Topics Covered:

- CSS Flexbox
- CSS Grid
- CSS Box Model (Margin/Padding/Border/Box-sizing)
- CSS Pseudo-classes & Pseudo-elements
- CSS Responsive Design & Media Queries

Objective: Build a responsive pricing section with three plan cards, one visually highlighted as 'recommended'.

Task Description: A subscription product needs a pricing section comparing Basic, Pro, and Enterprise plans, with the Pro plan visually emphasized to guide user choice.

Functional Requirements:

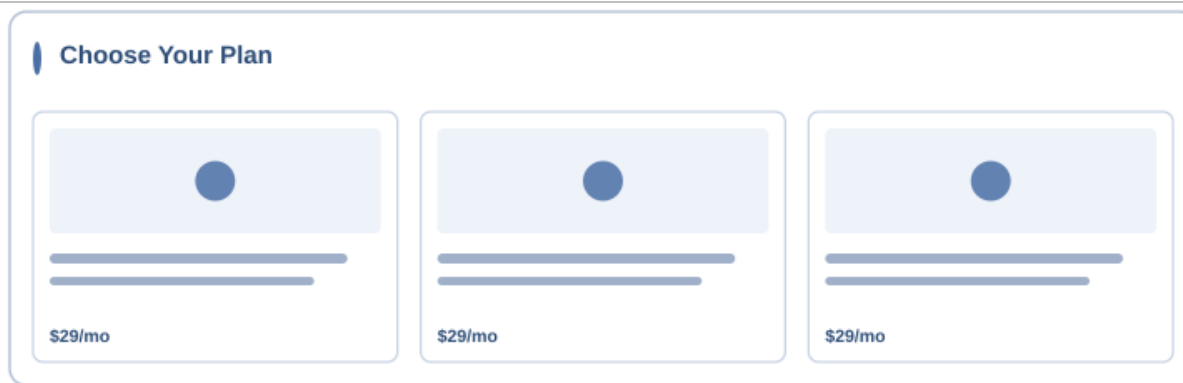
- Display 3 pricing cards side by side on desktop
- Each card lists a plan name, price, and at least 4 features
- The middle (recommended) card is visually emphasized (size, border, or color)
- Cards stack vertically on small screens

Technical Requirements: Use Flexbox or Grid for the card row, box-shadow/border for emphasis, :hover for interactivity, and a media query for stacking on mobile.

Expected Output: A pricing section where the recommended plan stands out clearly, and the layout adapts gracefully to mobile width.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshots at desktop and mobile widths

Acceptance Criteria:

- ☐ Three pricing cards are present with plan name, price, and features
- ☐ Recommended card is visually distinguishable from the others
- ☐ Cards stack correctly on narrow screens via a media query
- ☐ Hover feedback is present on cards or buttons
- ☐ No major functional issues

Task 14 — Blog Article Card Listing Page

Difficulty: **Medium**

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Grid
- CSS Pseudo-classes & Pseudo-elements
- CSS Text, Fonts & Units

Objective: Build a blog homepage that lists articles as cards in a responsive grid with hover interaction.

Task Description: A blogging platform needs a homepage listing recent articles as cards, each with a thumbnail placeholder, title, excerpt, and 'read more' link that responds to hover.

Functional Requirements:

- Display at least 6 article cards in a grid
- Each card shows a thumbnail placeholder, title, short excerpt, and a 'Read More' link
- Cards show a subtle hover effect (shadow, lift, or border change)
- Typography clearly distinguishes titles from excerpt text

Technical Requirements: Use CSS Grid for the card layout, pseudo-classes for hover interaction, and consistent text formatting rules for headings vs. body text.

Expected Output: A responsive blog grid where each card is clearly readable and provides visual feedback on hover.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ At least 6 article cards are displayed in a grid
- ☐ Each card includes thumbnail, title, excerpt, and read-more link
- ☐ Hover effect is visible and consistent across cards
- ☐ Grid reflows sensibly at reduced widths
- ☐ No major functional issues

Task 15 — Styled Registration Form with Inline JavaScript Validation

Difficulty: **Medium**

Topics Covered:

- HTML Forms & Input Elements
- CSS Styling of Lists/Tables/Forms/Images
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a registration form that checks required fields with JavaScript and shows inline feedback messages before allowing submission.

Task Description: An events website needs a sign-up form that gives users immediate feedback if they try to submit without filling in required fields, instead of relying only on a page reload.

Functional Requirements:

- Collect name, email, and password with clearly labeled fields
- On clicking Submit, check that no required field is empty

- Display an inline message next to any empty field indicating it is required
- Prevent the 'success' message from showing unless all fields are filled

Technical Requirements: Use HTML form elements with the required attribute as a baseline, plus JavaScript conditional statements, DOM selection, and event listeners to implement the custom validation feedback.

Expected Output: A form that clearly flags missing fields with inline messages and only confirms success once all fields are completed.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The image shows a light blue rounded rectangle containing a 'Sign Up' form. At the top left of the rectangle is a blue vertical bar followed by the text 'Sign Up'. Below this is the text 'Fields show inline validation messages'. The form itself is a white rounded rectangle with a light blue border. It contains three input fields, each with a label above it: 'FULL NAME', 'EMAIL', and 'PASSWORD'. Below the input fields is a blue button with the text 'CREATE ACCOUNT' in white.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing both an error state and a success state

Acceptance Criteria:

- ☐ All required fields are validated on submit
- ☐ Inline error messages appear next to empty fields
- ☐ Success message/state only appears when all fields are valid
- ☐ Validation logic uses JavaScript conditionals and DOM manipulation
- ☐ No major functional issues

Task 16 — Responsive Navbar with CSS Dropdown Submenu

Difficulty: Medium

Topics Covered:

- HTML Links, Navigation & Media
- CSS Flexbox
- CSS Display & Positioning
- CSS Pseudo-classes & Pseudo-elements

Objective: Build a navigation bar where one menu item reveals a dropdown submenu on hover.

Task Description: A company website needs a 'Services' menu item in its navbar that reveals a list of sub-services when hovered, using CSS only (no JavaScript).

Functional Requirements:

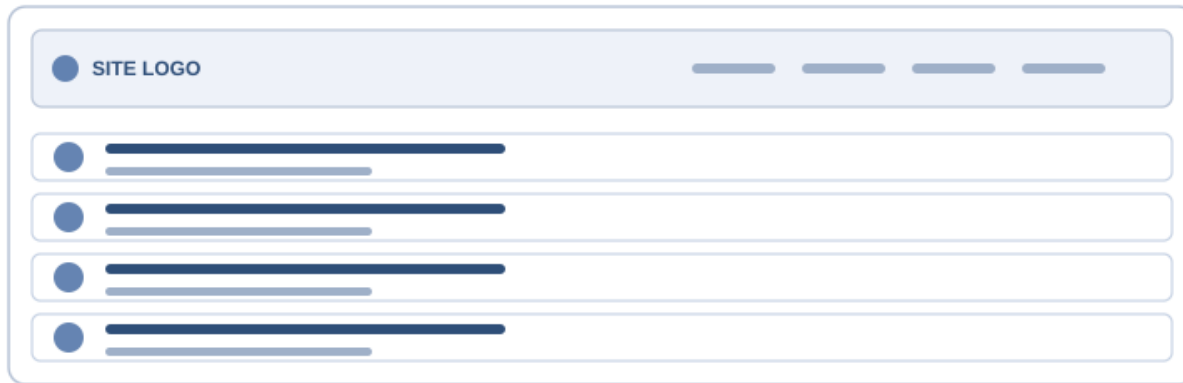
- Top-level navbar with at least 4 items, one labeled 'Services'
- Hovering 'Services' reveals a submenu with at least 4 sub-items
- Submenu is hidden by default and appears smoothly positioned under the parent item
- Other navbar items remain unaffected by the dropdown state

Technical Requirements: Use nested lists, CSS position (relative/absolute), display toggling on :hover, and z-index to layer the dropdown above page content.

Expected Output: A working hover-triggered dropdown submenu that appears directly beneath the 'Services' link and disappears when the mouse leaves.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshots showing closed and open dropdown states

Acceptance Criteria:

- ☐ Dropdown is hidden by default and appears only on hover
- ☐ Submenu is correctly positioned beneath its parent item
- ☐ z-index ensures the dropdown appears above other content
- ☐ Solution uses CSS only, no JavaScript
- ☐ No major functional issues

Task 17 — Image Gallery with Caption Overlay on Hover

Difficulty: Medium

Topics Covered:

- CSS Grid
- CSS Display & Positioning
- CSS Pseudo-classes & Pseudo-elements
- CSS Colors & Backgrounds

Objective: Build an image gallery where a semi-transparent caption overlay appears over each image on hover.

Task Description: A photography portfolio wants each gallery image to reveal a caption with the photo title when the visitor hovers over it, without needing extra clicks.

Functional Requirements:

- Display at least 6 images in a responsive grid
- Each image has a hidden caption overlay covering it
- Overlay becomes visible (fade/appear) when the image is hovered
- Overlay uses a semi-transparent background so the image remains partly visible

Technical Requirements: Use CSS Grid for gallery layout, position: absolute/relative for overlay placement, rgba() or opacity for transparency, and :hover to reveal the overlay.

Expected Output: A gallery grid where hovering any image smoothly reveals a captioned overlay directly on top of it.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshots showing normal and hovered states

Acceptance Criteria:

- ☐ At least 6 images are shown in a responsive grid
- ☐ Caption overlay is hidden by default on every image
- ☐ Hover reveals the overlay with a semi-transparent background
- ☐ Overlay is correctly positioned over its image using CSS positioning
- ☐ No major functional issues

Task 18 — Team & Testimonials Section

Difficulty: Medium

Topics Covered:

- CSS Grid

- CSS Flexbox
- CSS Box Model (Margin/Padding/Border/Box-sizing)
- CSS Text, Fonts & Units

Objective: Build a two-part section showing team member cards and customer testimonial cards.

Task Description: A company 'About Us' page needs a section introducing the team, followed by a testimonials section showing customer quotes with names and roles.

Functional Requirements:

- Team section shows at least 4 member cards (photo placeholder, name, role)
- Testimonials section shows at least 3 quote cards (quote text, name, role/company)
- Both sections use consistent card spacing and alignment
- Section headings clearly separate team members from testimonials

Technical Requirements: Use CSS Grid or Flexbox for both card rows, consistent box model spacing, and typography rules to differentiate quotes from names.

Expected Output: A clean two-section layout: an evenly spaced team grid followed by a testimonials grid, both easy to scan.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshot of final output

Acceptance Criteria:

- ☐ At least 4 team cards and 3 testimonial cards are present
- ☐ Both sections use consistent spacing via the box model
- ☐ Layout uses Flexbox or Grid appropriately for each section
- ☐ Section headings are clear and visually distinct
- ☐ No major functional issues

Task 19 — FAQ Accordion Panel

Difficulty: **Medium**

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Display & Positioning
- JS Events (Mouse & Keyboard)
- JS DOM Selection & Manipulation
- JS Conditional Statements

Objective: Build an FAQ section where clicking a question expands or collapses its answer.

Task Description: A support page needs an FAQ accordion so visitors can click on a question to reveal the answer, and click again to collapse it, keeping the page compact.

Functional Requirements:

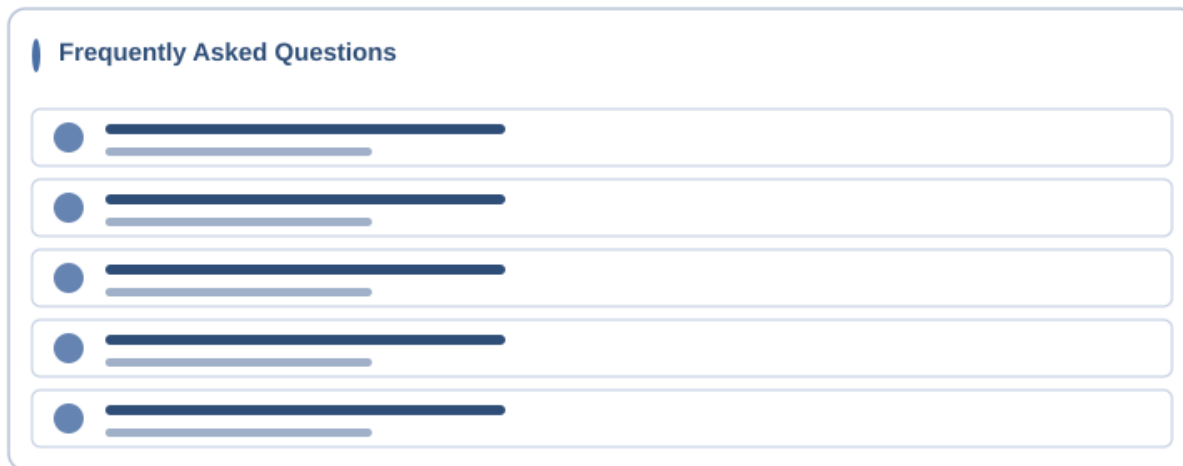
- Display at least 5 question/answer pairs
- Clicking a question toggles the visibility of its answer
- Only the clicked item's answer changes state (others remain unaffected, unless designed to close others)
- Visually indicate expanded vs. collapsed state (e.g., an icon or style change)

Technical Requirements: Use JavaScript click event listeners, DOM selection (`querySelectorAll`), conditional statements to check current state, and style/class changes to show or hide answers.

Expected Output: A working accordion where each question can be expanded and collapsed independently by clicking.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing collapsed and expanded states

Acceptance Criteria:

- ☐ At least 5 FAQ items are present
- ☐ Clicking a question correctly toggles its answer's visibility
- ☐ Expanded/collapsed state has a clear visual indicator

- ☐ Toggle logic is implemented with JavaScript events and DOM manipulation
- ☐ No major functional issues

Task 20 — Basic Arithmetic Calculator

Difficulty: **Medium**

Topics Covered:

- HTML Forms & Input Elements
- JS Operators & Type Coercion
- JS Functions & Objects
- JS Events (Mouse & Keyboard)
- JS DOM Selection & Manipulation

Objective: Build a simple on-page calculator that performs addition, subtraction, multiplication, and division on two numbers.

Task Description: A training exercise requires a small calculator tool where a user enters two numbers, chooses an operation, and sees the result instantly on the page.

Functional Requirements:

- Provide two number input fields
- Provide buttons (or a dropdown) for +, -, *, and / operations
- Display the calculated result on the page without reloading
- Handle division by zero with a clear message instead of an error

Technical Requirements: Use JavaScript arithmetic operators, a function per operation (or a shared function with conditionals), event listeners on buttons, and DOM updates to show the result.

Expected Output: A functioning calculator where selecting an operation instantly displays the correct computed result for the entered numbers.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a light blue rounded rectangle containing the title 'Calculator' with a vertical bar icon to its left. Below the title is a white rounded rectangle with two input fields labeled 'NUMBER A' and 'NUMBER B'. Under 'NUMBER A' is a blue 'CALCULATE' button.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot of final output showing a calculation result

Acceptance Criteria:

- ☐ All four operations produce correct results
- ☐ Division by zero is handled gracefully with a message

- ☐ Result updates on the page without a reload
 - ☐ Input values are correctly read and converted for calculation
 - ☐ No major functional issues
-

Task 21 — Login Form with JavaScript Empty-Field Validation

Difficulty: **Medium**

Topics Covered:

- HTML Forms & Input Elements
- JS Conditional Statements
- JS Events (Mouse & Keyboard)
- JS DOM Selection & Manipulation

Objective: Build a login form that prevents submission and shows a warning message when either field is left empty.

Task Description: An internal tool needs a simple login screen that blocks submission and clearly warns the user if the username or password field is left blank.

Functional Requirements:

- Provide username and password fields plus a login button
- On clicking Login, check both fields for empty values
- Show a clear warning message if one or both fields are empty
- Show a distinct success message/state if both fields are filled

Technical Requirements: Use JavaScript conditional statements (if/else) to check field values, DOM selection to read input values, and DOM manipulation to display the message.

Expected Output: A login form that reliably blocks and warns on empty fields, and confirms success only when both fields contain input.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The image shows a structural mockup of a login form. It consists of a light blue rounded rectangle containing the following elements: a title 'Login' with a blue icon to its left; a 'USERNAME' label above a text input field; a 'PASSWORD' label above a password input field; and a blue 'LOGIN' button at the bottom.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing error and success states

Acceptance Criteria:

- ☐ Empty username or password triggers a visible warning
 - ☐ Warning message is clear and does not overlap other content
 - ☐ Success state only appears when both fields are filled
 - ☐ Validation logic correctly uses conditionals and DOM access
 - ☐ No major functional issues
-

Task 22 — Product Listing Page with Category Filter Buttons

Difficulty: **Medium**

Topics Covered:

- CSS Grid
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Conditional Statements
- JS Loops

Objective: Build a product grid where clicking a category button shows only matching products using JavaScript.

Task Description: An online store's category page should let visitors filter a grid of products by clicking category buttons (e.g., All, Clothing, Electronics, Accessories), without reloading the page.

Functional Requirements:

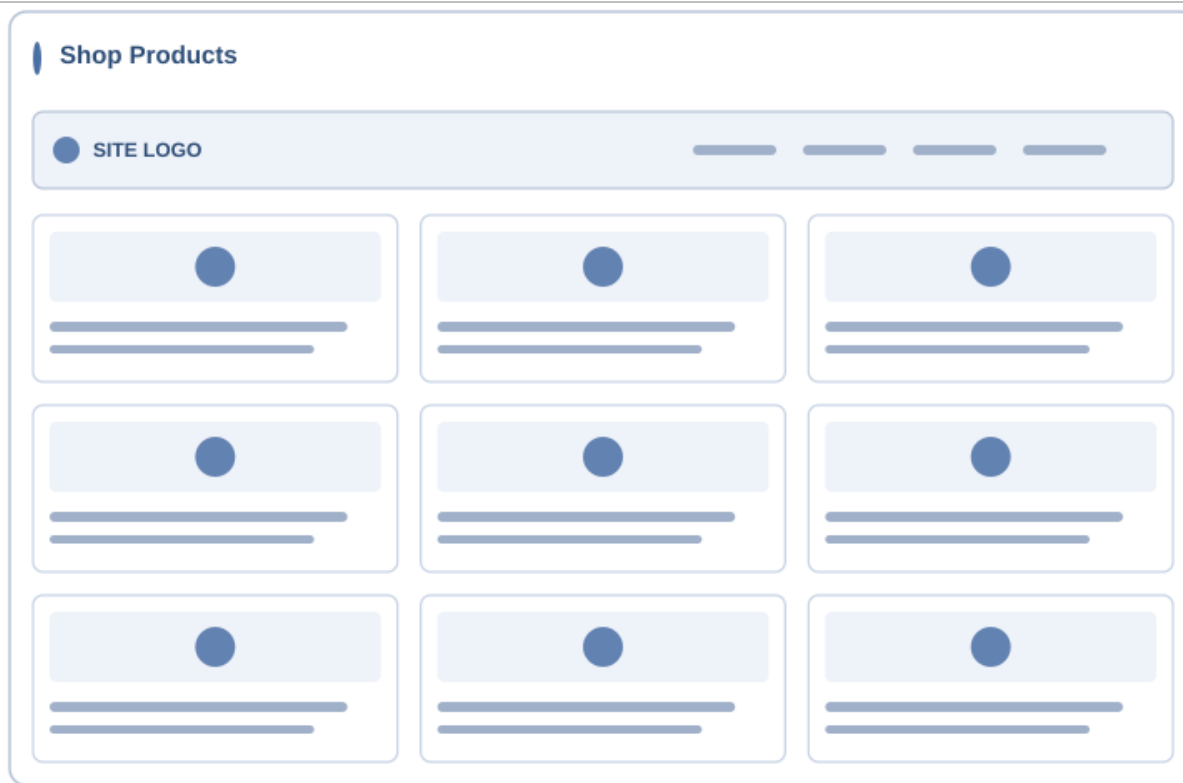
- Display at least 9 products in a grid, each tagged with a category
- Provide filter buttons for 'All' plus at least 3 categories
- Clicking a category button shows only products from that category
- Clicking 'All' restores the full product grid

Technical Requirements: Use JavaScript to loop through product elements, compare each product's category attribute using conditionals, and show/hide items via DOM style/class changes triggered by click events.

Expected Output: A responsive product grid that instantly filters to show only the selected category when a filter button is clicked.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing at least two different filtered states

Acceptance Criteria:

- ☐ At least 9 products across at least 3 categories are present
- ☐ Each filter button correctly shows only matching products
- ☐ 'All' button correctly restores the full grid
- ☐ Filtering logic uses loops, conditionals, and DOM manipulation in JavaScript
- ☐ No major functional issues

Task 23 — Interactive Quiz Application with Score Tracking

Difficulty: Medium

Topics Covered:

- HTML Structure & Semantic Elements
- JS Variables, Data Types & Scope
- JS Functions & Objects
- JS Loops
- JS Conditional Statements
- JS Events (Mouse & Keyboard)

Objective: Build a simple multiple-choice quiz that tracks and displays the user's score at the end.

Task Description: A training portal needs a short quiz feature: the user answers a series of multiple-choice questions, and their final score is shown once they finish.

Functional Requirements:

- Present at least 5 multiple-choice questions, one at a time or as a list
- Allow the user to select an answer for each question
- Track the number of correct answers as the user progresses
- Display the final score (e.g., '4 out of 5') at the end

Technical Requirements: Store questions/answers in JavaScript objects, use a loop to process them, conditionals to check correctness, a variable to track score, and event listeners to capture answer selections.

Expected Output: A working quiz flow that ends with an accurate score summary based on the user's selected answers.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot of a question in progress and the final score screen

Acceptance Criteria:

- ☐ At least 5 questions are presented with selectable answers
- ☐ Score correctly increments only for correct answers
- ☐ Final score is displayed accurately at the end
- ☐ Quiz data and logic are organized using JavaScript objects, loops and conditionals
- ☐ No major functional issues

Task 24 — Two-Column Blog Page with Sticky Sidebar

Difficulty: **Medium**

Topics Covered:

- CSS Flexbox
- CSS Display & Positioning

- CSS Responsive Design & Media Queries
- CSS Text, Fonts & Units
- CSS Math Functions (calc/min/max/clamp)

Objective: Build a blog article page with a main content column and a sticky sidebar that stays visible while scrolling.

Task Description: A blog wants article pages where the sidebar (author info, recent posts, categories) stays in view as the reader scrolls through a long article, and collapses below the article on mobile.

Functional Requirements:

- Main column displays an article with multiple paragraphs and subheadings
- Sidebar displays author info, a recent-posts list, and a category list
- Sidebar remains visible ('sticky') while the main content scrolls, on desktop
- On small screens, sidebar moves below the article in a single column

Technical Requirements: Use Flexbox for the two-column layout, position: sticky for the sidebar, the clamp() math function for fluid article heading sizes, and a media query to switch to a single-column stacked layout on small screens.

Expected Output: A two-column article layout where the sidebar sticks in view on desktop and stacks neatly below content on mobile.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshots at desktop and mobile widths

Acceptance Criteria:

- ☐ Main content and sidebar are laid out in two columns on desktop
- ☐ Sidebar uses position: sticky and remains visible while scrolling
- ☐ Layout switches to a single column on small screens via media query
- ☐ Typography is readable and consistent throughout
- ☐ No major functional issues

Task 25 — Live Character Counter & Field Validator for a Feedback Form

Difficulty: **Medium**

Topics Covered:

- HTML Forms & Input Elements
- JS Events (Mouse & Keyboard)
- JS DOM Selection & Manipulation
- JS Conditional Statements
- JS Operators & Type Coercion

Objective: Build a feedback form with a message box limited to 200 characters, showing a live counter and blocking over-limit submissions.

Task Description: A customer feedback form needs a message box with a strict 200-character limit. Users should see how many characters remain as they type, with a clear warning once they exceed the limit.

Functional Requirements:

- Textarea for feedback with a visible '0 / 200' style live counter
- Counter updates on every keystroke as the user types
- Counter (or textarea) visually indicates an error state when the limit is exceeded
- Submit button is disabled while the message exceeds 200 characters

Technical Requirements: Use a keyboard/input event listener, JavaScript string length comparisons with conditionals, and DOM manipulation to update the counter text and toggle the disabled state of the submit button.

Expected Output: A feedback form where the character counter updates live while typing and the submit button becomes disabled if the limit is exceeded.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

A structural reference mockup of a feedback form. It consists of a light blue rounded rectangle containing a header 'Send Feedback' with a blue icon. Below the header is a text input field with a placeholder 'YOUR MESSAGE (0 / 200)'. At the bottom left of the form is a blue 'SEND' button.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing normal and over-limit states

Acceptance Criteria:

- ☐ Counter accurately reflects the current character count at all times
- ☐ Exceeding 200 characters triggers a visible warning state
- ☐ Submit button is correctly disabled/enabled based on character count
- ☐ Logic is implemented using JavaScript events, conditionals, and DOM manipulation
- ☐ No major functional issues

3. Hard Tasks

Task 26 – Task 50 | Realistic, multi-component projects with complex functionality.

Task 26 — Multi-Page Business Website with Shared Responsive Navigation

Difficulty: Hard

Topics Covered:

- HTML Structure & Semantic Elements
- HTML Links, Navigation & Media
- CSS Flexbox
- CSS Responsive Design & Media Queries
- CSS Pseudo-classes & Pseudo-elements
- CSS Cascade, Specificity & Inheritance

Objective: Build a 4-page business website (Home, About, Services, Contact) sharing a consistent responsive navigation and footer.

Task Description: A small consultancy needs a proper multi-page website rather than a single scrolling page, with consistent branding, navigation, and footer across every page, and a working contact form on the Contact page.

Functional Requirements:

- Separate HTML pages for Home, About, Services, and Contact, linked via a shared navigation bar
- Home page includes a hero section and a summary of services
- About page includes company story and team section
- Services page lists services in a card or grid layout
- Contact page includes a working contact form (name, email, message)
- Active page is visually indicated in the navigation

Technical Requirements: Use consistent HTML/CSS structure across pages, Flexbox for navigation and layout, media queries for responsiveness, and a shared external stylesheet to keep styling consistent (demonstrating CSS cascade).

Expected Output: Four linked, consistently styled pages that behave like a real small-business website, fully navigable and responsive.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (all HTML pages, shared CSS file)
- Screenshots of each page
- Screenshot of mobile navigation

Acceptance Criteria:

- ☐ All 4 pages exist and are linked correctly from a shared navigation bar
- ☐ Styling is visually consistent across all pages (shared stylesheet)
- ☐ Active/current page is indicated in the navigation
- ☐ Layout is responsive on at least 2 screen sizes
- ☐ Contact form on the Contact page is present and well structured
- ☐ No major functional or navigation issues

Task 27 — E-Commerce Product Catalogue with Live Cart Counter

Difficulty: Hard

Topics Covered:

- CSS Grid
- CSS Flexbox
- CSS Responsive Design & Media Queries
- JS Variables, Data Types & Scope
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Operators & Type Coercion

Objective: Build a responsive product catalogue where an 'Add to Cart' button on each product updates a running cart counter and total in the navbar.

Task Description: An online store catalogue page should let shoppers browse products in a responsive grid and add items to a cart, seeing the item count and running total update instantly in the header.

Functional Requirements:

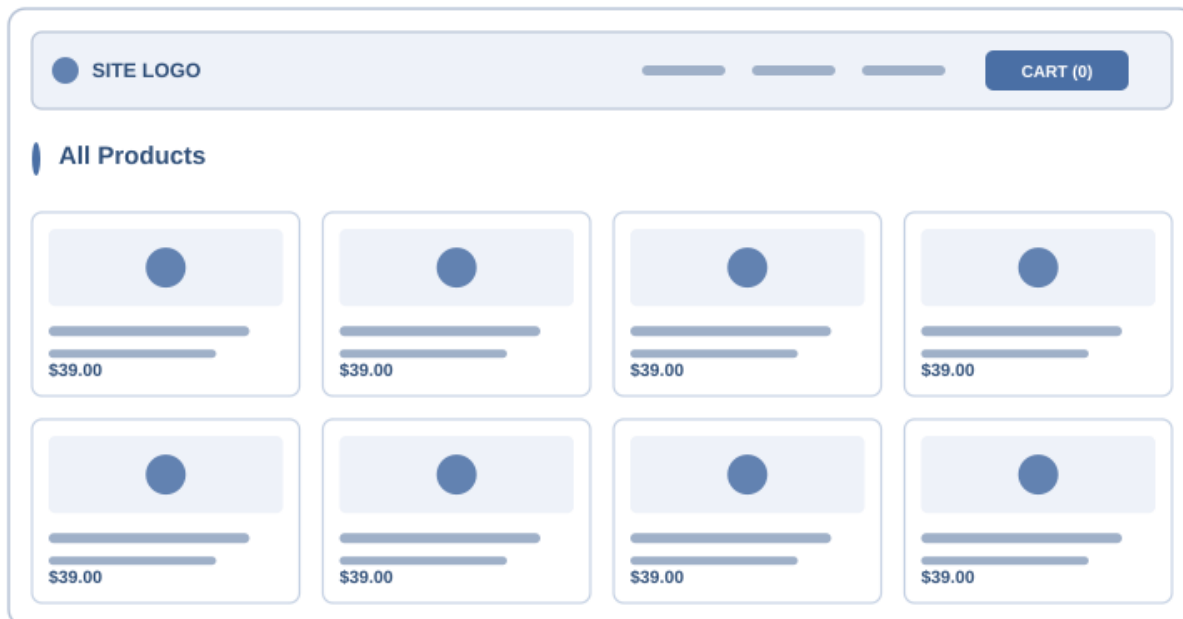
- Display at least 8 products with image placeholder, name, price, and 'Add to Cart' button
- Navbar shows a live cart icon/badge with item count
- Navbar or a cart summary area shows a running total price
- Grid layout adapts from multiple columns on desktop to fewer on mobile

Technical Requirements: Use CSS Grid for the catalogue layout and a media query for responsiveness. Use JavaScript variables to track cart count/total, event listeners on each 'Add to Cart' button, and DOM updates to reflect the new count/total.

Expected Output: A responsive product grid where clicking 'Add to Cart' on any product immediately updates the cart badge count and total shown in the header.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing the cart before and after adding items

Acceptance Criteria:

- ☐ At least 8 products are displayed with correct details
- ☐ Cart count updates correctly for every 'Add to Cart' click
- ☐ Running total price is calculated and displayed correctly
- ☐ Grid layout is responsive across at least two breakpoints
- ☐ Cart logic uses JavaScript variables, events, and arithmetic operators correctly
- ☐ No major functional issues

Task 28 — Admin Dashboard UI with Sidebar Navigation, Stat Cards & Data Table

Difficulty: Hard

Topics Covered:

- CSS Flexbox
- CSS Grid
- CSS Display & Positioning
- HTML Tables
- CSS Styling of Lists/Tables/Forms/Images
- CSS Responsive Design & Media Queries

Objective: Build a static admin dashboard interface with a sidebar, summary stat cards, and a data table.

Task Description: An internal admin panel needs a dashboard home screen showing a fixed sidebar for navigation, a row of key metric cards at the top, and a recent-activity data table below.

Functional Requirements:

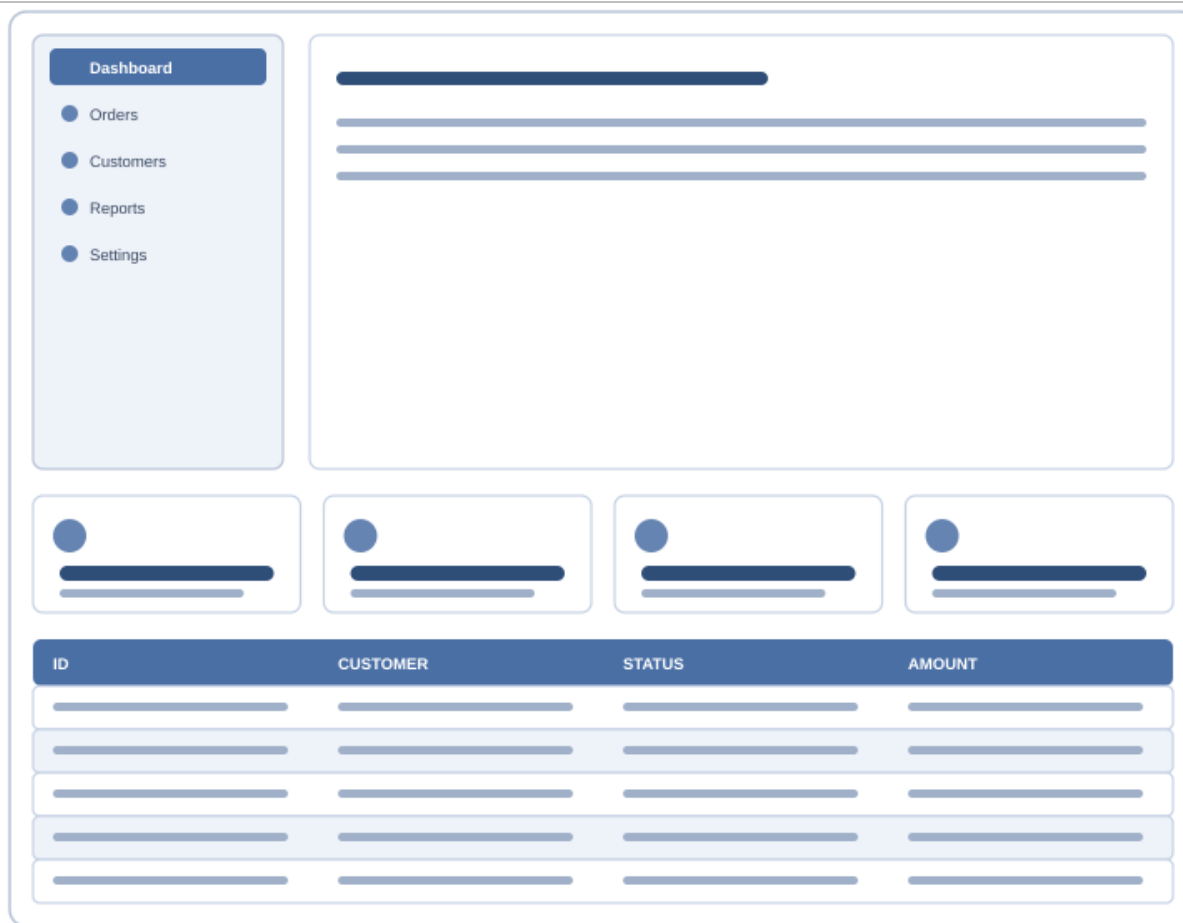
- Left sidebar with at least 5 navigation items and an active-item highlight
- Top row of at least 4 stat cards (e.g., Total Users, Revenue, Orders, Visits)
- Data table listing at least 6 rows of recent activity/records with a header row
- Layout adapts so the sidebar collapses or moves on smaller screens

Technical Requirements: Use Flexbox or Grid for the overall dashboard layout, CSS Grid for the stat cards row, semantic table markup with styling (zebra rows, header shading), and a media query for the responsive sidebar behavior.

Expected Output: A dashboard layout resembling a real admin panel: sidebar, stat cards, and a clean, readable data table, all responsive.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS files)
- Screenshots at desktop and mobile widths

Acceptance Criteria:

- ☐ Sidebar, stat cards, and data table are all present and correctly structured
- ☐ Active navigation item is visually indicated in the sidebar
- ☐ Stat cards are arranged in a clean, evenly spaced row/grid
- ☐ Data table is legible with header styling and zebra striping
- ☐ Layout is responsive with sidebar behavior adjusting on small screens
- ☐ No major functional issues

Task 29 — Hotel Room Booking Interface with Price Summary Calculator

Difficulty: Hard

Topics Covered:

- HTML Forms & Input Elements
- CSS Grid
- CSS Flexbox
- JS Operators & Type Coercion

- JS Functions & Objects
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a hotel booking page where selecting room type and number of nights calculates and displays a live price summary.

Task Description: A boutique hotel wants an online room-booking page: guests pick a room type and enter check-in/check-out details, and the page calculates the total cost live before they submit an enquiry.

Functional Requirements:

- Display at least 3 room type cards with nightly rates
- Provide a booking form capturing check-in date, check-out date (or number of nights), and guest count
- Automatically calculate and display the total price based on selected room and number of nights
- Show a booking summary before final submission

Technical Requirements: Use CSS Grid/Flexbox for room cards, HTML date/number inputs, and JavaScript (variables, arithmetic operators, functions, DOM manipulation, event listeners) to compute and display the live total.

Expected Output: A booking interface where choosing a room and entering stay details instantly shows an accurate calculated total in a summary panel.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a booking interface titled "Choose Your Room". It features three room cards, each with a blue circle icon, a horizontal bar, and the text "\$120/night". Below the cards is a form section with labels "CHECK-IN", "CHECK-OUT", and "GUESTS", each followed by an input field. A blue button labeled "CALCULATE TOTAL" is positioned at the bottom left of the form section.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing a completed price summary

Acceptance Criteria:

- ☐ At least 3 room types with rates are displayed
- ☐ Booking form correctly captures dates/nights and guest count
- ☐ Total price calculation is accurate based on selected inputs
- ☐ Summary updates live as the user changes selections

- ☐ Calculation logic correctly uses JavaScript functions and operators
 - ☐ No major functional issues
-

Task 30 — Movie Ticket Seat Selection Interface

Difficulty: Hard

Topics Covered:

- CSS Grid
- CSS Pseudo-classes & Pseudo-elements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Conditional Statements
- JS Loops
- JS Operators & Type Coercion

Objective: Build an interactive cinema seat map where users click to select seats and see a live running total.

Task Description: A cinema booking page needs a seat map (rows x columns) where each seat can be clicked to toggle selection, already-booked seats are disabled, and the total ticket price updates live as seats are selected.

Functional Requirements:

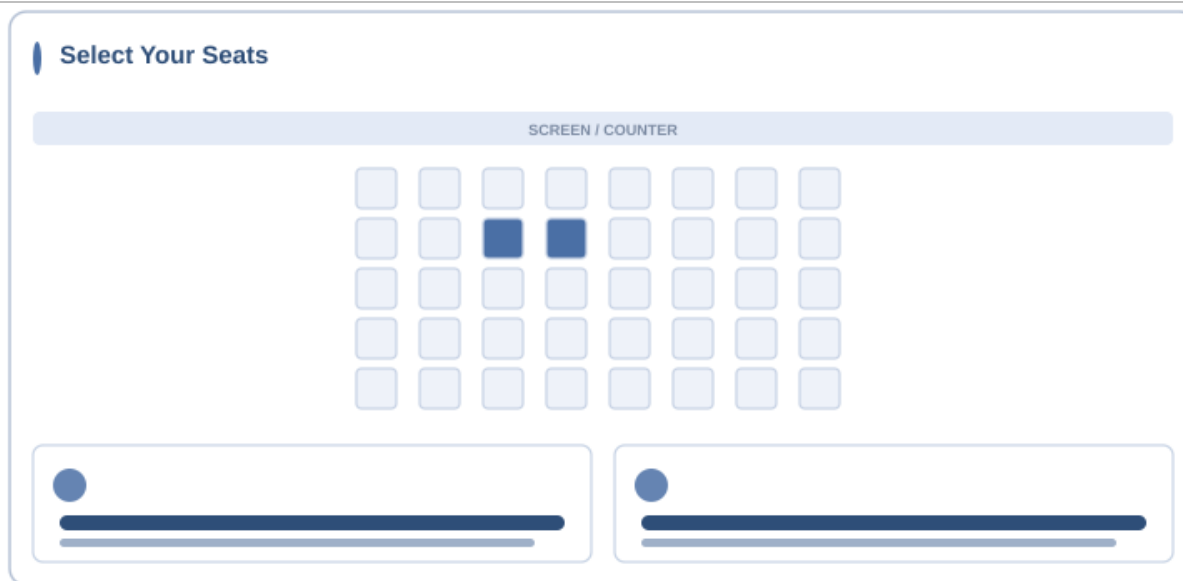
- Render a grid of seats (e.g., 5 rows x 8 columns) representing the screen layout
- Some seats are pre-marked as already booked and cannot be selected
- Clicking an available seat toggles it between selected and unselected
- A running total price and selected-seat count update live as seats are chosen

Technical Requirements: Use CSS Grid to render the seat map and pseudo-classes for visual states. Use JavaScript loops to generate/manage seats, click event listeners, conditionals to check seat status, and DOM manipulation plus arithmetic operators to update price/count.

Expected Output: An interactive seat map where clicking available seats correctly toggles selection and keeps the price/count summary accurate at all times.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing selected, available, and booked seat states

Acceptance Criteria:

- ☐ Seat grid correctly renders with distinct states for available, selected, and booked seats
- ☐ Booked seats cannot be selected
- ☐ Clicking a seat correctly toggles its selected state
- ☐ Running total and seat count are accurate at all times
- ☐ Logic is implemented using JavaScript loops, conditionals, and DOM manipulation
- ☐ No major functional issues

Task 31 — Online Food Delivery Menu with Cart Total Calculation

Difficulty: Hard

Topics Covered:

- CSS Grid
- CSS Flexbox
- JS Variables, Data Types & Scope
- JS Functions & Objects
- JS Operators & Type Coercion
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Loops

Objective: Build a food delivery menu page where users add items to a cart and see a live itemized order summary with total.

Task Description: A restaurant's food delivery page should list menu items by category, let users add items to a cart with quantity control, and show a running order summary with subtotal, a fixed delivery fee, and grand total.

Functional Requirements:

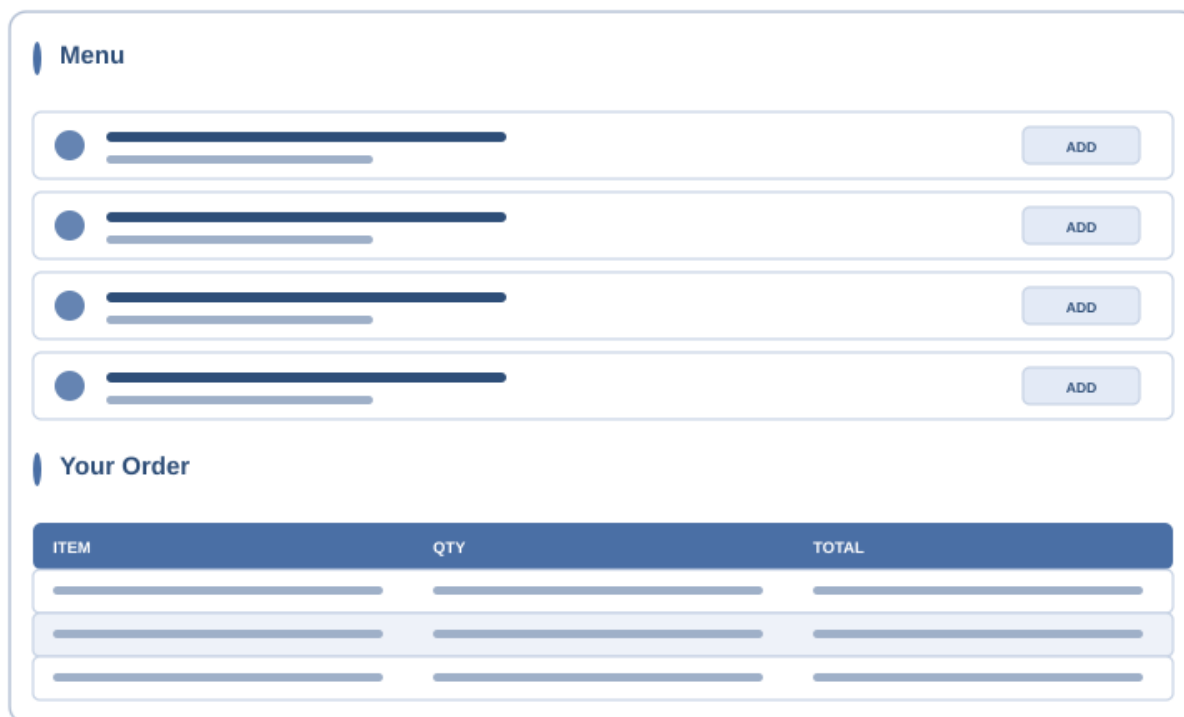
- Display menu items grouped into at least 3 categories with name, price, and an 'Add' control
- Cart/order summary panel lists added items with quantity and line total
- Provide quantity increase/decrease controls per cart item
- Automatically calculate subtotal, add a fixed delivery fee, and display the grand total

Technical Requirements: Use CSS Grid/Flexbox for the menu and summary panel. Use JavaScript objects/variables to track cart items, loops to render/update the cart, arithmetic operators for totals, and event listeners for add/quantity controls.

Expected Output: A two-panel food ordering interface where the cart summary accurately reflects every add/quantity change, ending in a correct grand total.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing a populated cart with subtotal, fee, and total

Acceptance Criteria:

- ☐ Menu items are grouped into at least 3 categories
- ☐ Adding an item correctly updates the cart summary
- ☐ Quantity controls correctly adjust line totals and subtotal
- ☐ Grand total correctly includes the delivery fee
- ☐ Cart logic correctly uses JavaScript objects, loops, and operators
- ☐ No major functional issues

Task 32 — Job Listings Portal with Multi-Filter Search Buttons

Difficulty: Hard

Topics Covered:

- CSS Grid
- HTML Forms & Input Elements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Conditional Statements
- JS Loops

Objective: Build a job listings page where users can filter listings by category and job type using clickable filter buttons.

Task Description: A job portal listing page needs to let candidates narrow down job cards by category (e.g., Design, Engineering, Marketing) and by job type (Full-time, Part-time, Remote) using instant client-side filtering.

Functional Requirements:

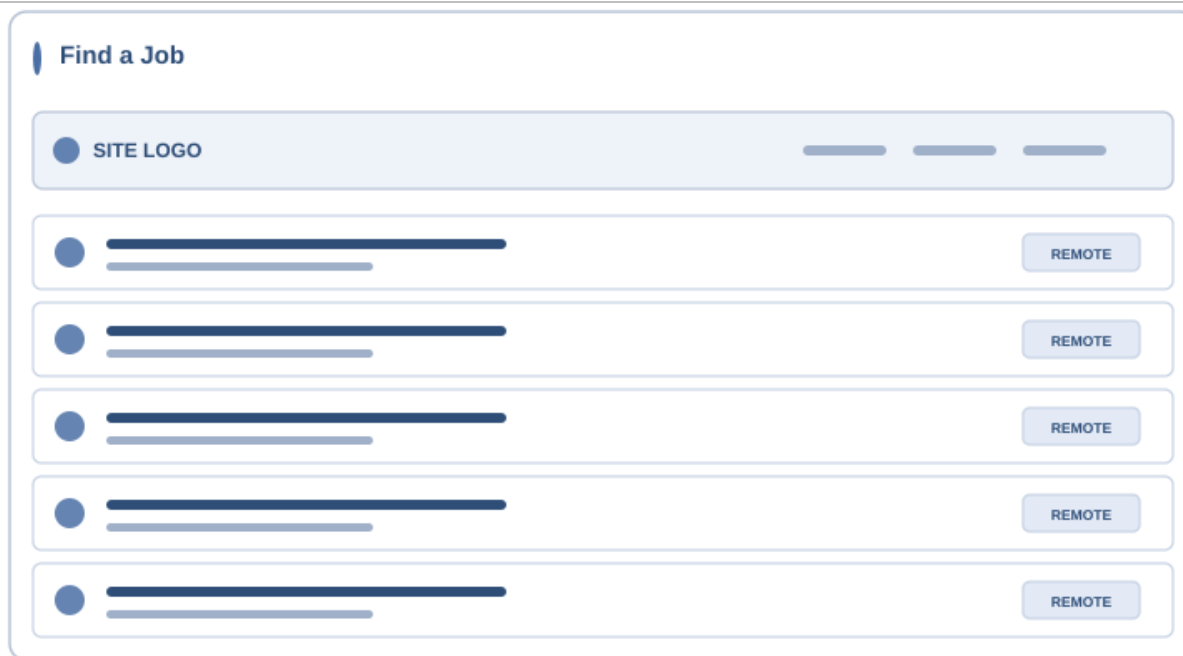
- Display at least 10 job cards with title, company, category, type, and location
- Provide category filter buttons and job-type filter buttons
- Filtering by category and type can be combined (e.g., Engineering + Remote)
- Show a friendly message when no jobs match the selected filters

Technical Requirements: Use CSS Grid for the job card layout. Use JavaScript loops to iterate job cards, conditionals to test multiple filter criteria together, and DOM manipulation triggered by click events to show/hide matching cards.

Expected Output: A job listings grid that accurately filters to show only jobs matching the combination of selected category and type filters.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing at least two different filter combinations

Acceptance Criteria:

- ☐ At least 10 job cards are displayed with correct attributes
- ☐ Category and type filters each work correctly on their own
- ☐ Combined filtering (category + type) produces the correct results
- ☐ A no-results message appears when no jobs match
- ☐ Filtering logic uses JavaScript loops, conditionals, and DOM manipulation
- ☐ No major functional issues

Task 33 — Real Estate Property Listing Website

Difficulty: Hard

Topics Covered:

- HTML Structure & Semantic Elements
- HTML Links, Navigation & Media
- CSS Grid
- CSS Responsive Design & Media Queries
- HTML Forms & Input Elements
- CSS Cascade, Specificity & Inheritance

Objective: Build a two-page real estate site: a property listing page and a property detail page, linked together.

Task Description: A real estate agency needs a listings page showing available properties as cards, and a separate detail page (used as a template) showing full details of one property, with an enquiry form.

Functional Requirements:

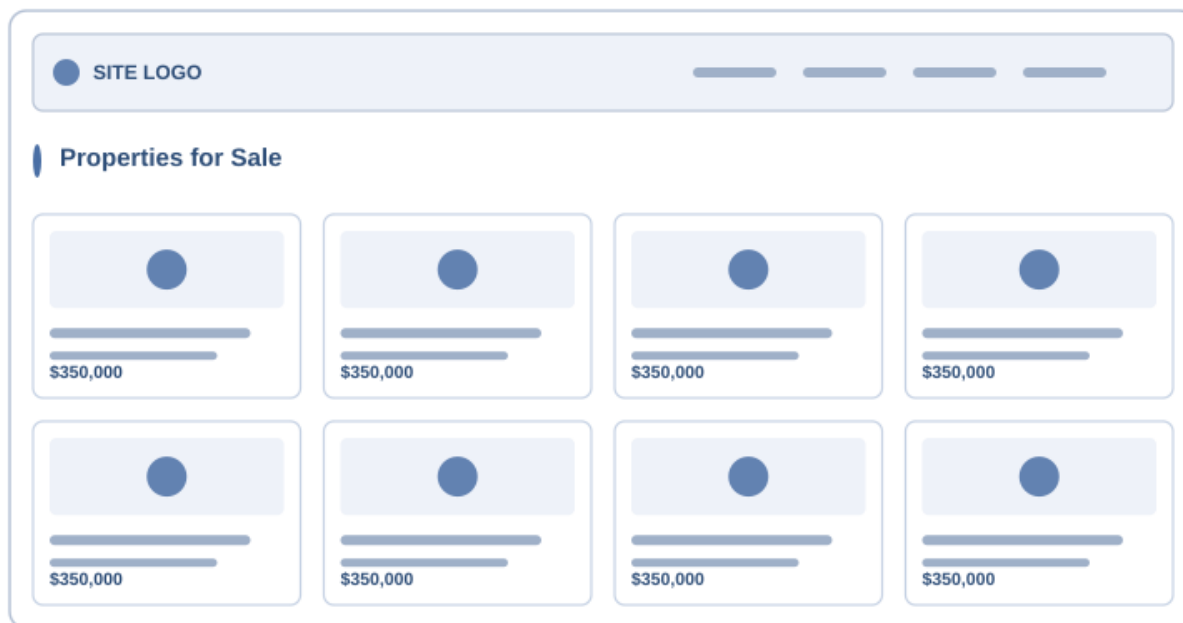
- Listings page displays at least 8 property cards (image placeholder, price, location, bedrooms)
- Clicking a property card links to the property detail page
- Detail page shows a larger image placeholder, full description, feature list, and an enquiry form
- Both pages share a consistent header, navigation, and footer

Technical Requirements: Use CSS Grid for the listings layout, HTML anchor links between pages, HTML forms for the enquiry section, and a shared stylesheet to maintain visual consistency (cascade) across both pages.

Expected Output: A working two-page flow where browsing the listings page and clicking a property leads to a consistent, detailed property page.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (both HTML pages, shared CSS file)
- Screenshots of both pages

Acceptance Criteria:

- ☐ Listings page shows at least 8 properties in a responsive grid
- ☐ Each property card correctly links to the detail page
- ☐ Detail page includes full description, features, and an enquiry form
- ☐ Both pages share consistent header/footer/styling
- ☐ Layout is responsive at multiple widths
- ☐ No major functional or navigation issues

Task 34 — Fitness Class Booking Schedule with Interactive Slot Selection

Difficulty: Hard

Topics Covered:

- HTML Tables

- HTML Forms & Input Elements
- CSS Styling of Lists/Tables/Forms/Images
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Conditional Statements

Objective: Build a weekly class schedule table where clicking an available slot opens a booking form for that specific class.

Task Description: A gym wants a weekly schedule (days as columns, time slots as rows) where members can click on any open class slot to bring up a small booking form pre-filled with that class's details.

Functional Requirements:

- Display a schedule table with at least 5 days and at least 4 time-slot rows
- Some slots show a class name (bookable), others show 'Closed' (not bookable)
- Clicking a bookable slot reveals a booking form showing the class name, day, and time
- Clicking a closed slot does nothing or shows an appropriate message

Technical Requirements: Use an HTML table for the schedule with CSS styling for open vs. closed cells. Use JavaScript event listeners on table cells, conditionals to check slot status, and DOM manipulation to populate and reveal the booking form.

Expected Output: A schedule table where only bookable class slots respond to clicks, correctly opening a pre-filled booking form for that slot.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a 'Weekly Class Schedule' interface. At the top, there's a title 'Weekly Class Schedule' with a vertical bar icon. Below it is a table with 5 columns representing days of the week: MON, TUE, WED, THU, and FRI. Each column has 4 rows representing time slots. The first row of each column is highlighted in a darker blue, while the other three rows are lighter. Below the table is a booking form. It has two input fields: 'CLASS' and 'DAY & TIME'. Below these fields is a blue button labeled 'CONFIRM BOOKING'.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots of the schedule and an opened booking form

Acceptance Criteria:

- ☐ Schedule table correctly shows bookable and closed slots with distinct styling
- ☐ Clicking a bookable slot correctly opens a booking form with correct pre-filled details

- ☐ Clicking a closed slot does not open a booking form
 - ☐ Table and form styling are clean and legible
 - ☐ Logic correctly uses JavaScript conditionals, events, and DOM manipulation
 - ☐ No major functional issues
-

Task 35 — E-Learning Course Catalogue with Enrollment Form

Difficulty: Hard

Topics Covered:

- CSS Grid
- CSS Flexbox
- HTML Forms & Input Elements
- CSS Responsive Design & Media Queries
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a course catalogue page with course cards and a modal-style enrollment form triggered from each card.

Task Description: An online learning platform needs a catalogue page listing available courses, where clicking 'Enroll' on any course opens an enrollment form (as an overlay panel) pre-filled with the course name.

Functional Requirements:

- Display at least 8 course cards with title, instructor, duration, and level
- Each card has an 'Enroll' button
- Clicking 'Enroll' opens an overlay/panel enrollment form pre-filled with that course's name
- The overlay can be closed to return to browsing the catalogue

Technical Requirements: Use CSS Grid for the catalogue and CSS positioning (fixed/absolute) for the overlay form. Use JavaScript DOM manipulation and event listeners to open/close the overlay and pre-fill the course name field.

Expected Output: A course catalogue where clicking 'Enroll' on any card reliably opens a correctly pre-filled enrollment overlay, closable without page reload.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

Explore Courses

COURSE NAME

YOUR EMAIL

CONFIRM ENROLLMENT

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots of the catalogue and an open enrollment overlay

Acceptance Criteria:

- ☐ At least 8 course cards are displayed with complete details
- ☐ Enroll button correctly opens the overlay form for the correct course
- ☐ Course name field in the overlay is correctly pre-filled
- ☐ Overlay can be closed and reopened correctly
- ☐ Grid layout is responsive across screen sizes
- ☐ No major functional issues

Task 36 — Multi-Section Job Application Form with Step Validation

Difficulty: Hard

Topics Covered:

- HTML Forms & Input Elements
- CSS Styling of Lists/Tables/Forms/Images
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Loops

Objective: Build a multi-section job application form (Personal Info, Experience, Skills) where each section is validated before the user can proceed to the next.

Task Description: A recruitment platform needs a longer application form broken into clear sections, so candidates fill and confirm one section at a time, reducing errors on final submission.

Functional Requirements:

- Section 1: Personal details (name, email, phone)
- Section 2: Experience details (previous role, years of experience)
- Section 3: Skills (checkboxes for at least 5 skills)
- 'Next' button in each section is disabled/blocked until required fields in that section are valid
- A final review/summary is shown before submission

Technical Requirements: Use HTML form sections styled with CSS, JavaScript conditionals to validate each section's fields, loops to check checkbox groups, event listeners for navigation buttons, and DOM manipulation to show/hide sections and the summary.

Expected Output: A guided, section-by-section application form that only allows progression once each section's required fields are correctly filled, ending in an accurate summary.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a form with three tabs: 'Personal', 'Experience', and 'Skills'. The 'Personal' tab is selected. Below the tabs, there are two horizontal bars. The main form area contains three input fields labeled 'FULL NAME', 'EMAIL', and 'PHONE'. At the bottom of this section is a blue 'NEXT' button.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots of each section and the final summary

Acceptance Criteria:

- ☐ All three sections are present with correct fields
- ☐ User cannot proceed to the next section with invalid/empty required fields
- ☐ Skill checkboxes are correctly validated (e.g., at least one selected)
- ☐ Final summary accurately reflects all entered data
- ☐ Section navigation logic correctly uses JavaScript conditionals and DOM manipulation
- ☐ No major functional issues

Task 37 — Personal Expense Tracker with Running Total

Difficulty: Hard

Topics Covered:

- HTML Forms & Input Elements
- HTML Tables
- JS Variables, Data Types & Scope
- JS Functions & Objects
- JS Loops
- JS Operators & Type Coercion
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build an expense tracker where users add expense entries via a form and see them listed in a table with a live running total.

Task Description: A personal finance tool needs a simple expense tracker: users enter an expense description, category, and amount; the entry is added to a table, and the total spent updates automatically.

Functional Requirements:

- Form to add a new expense (description, category, amount)
- New entries appear immediately in an expense table without a page reload
- Running total of all expenses is displayed and updates with each new entry
- Provide a 'Delete' action per row that removes the entry and updates the total

Technical Requirements: Use an HTML form and table. Use JavaScript objects/arrays to store expenses, loops to render the table, arithmetic operators to compute the total, and event listeners for adding and deleting entries with DOM manipulation.

Expected Output: A working expense tracker where adding or deleting entries instantly and correctly updates both the table and the running total.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

DESCRIPTION

CATEGORY

AMOUNT

ADD EXPENSE

DESCRIPTION	CATEGORY	AMOUNT	ACTION

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing multiple entries and the updated total

Acceptance Criteria:

- ☐ New expenses are correctly added to the table without a page reload
- ☐ Running total correctly reflects all current entries
- ☐ Deleting an entry correctly removes it and updates the total
- ☐ Data is managed using JavaScript objects/arrays and loops
- ☐ Form input is validated so empty/invalid entries are not added
- ☐ No major functional issues

Task 38 — Hospital Appointment Booking System Interface

Difficulty: **Hard**

Topics Covered:

- HTML Forms & Input Elements
- HTML Tables
- CSS Grid
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a hospital appointment booking interface where patients select a department and doctor, then submit a validated booking form.

Task Description: A hospital's patient portal needs an appointment page: patients browse doctors by department, then fill out a booking form with date/time preferences, which is validated before showing a confirmation.

Functional Requirements:

- Display doctor cards grouped by at least 3 departments (name, specialty, available days)
- Selecting a doctor pre-fills the booking form with their name and department
- Booking form validates that date, time, and patient details are all provided
- Show a confirmation summary table after successful submission

Technical Requirements: Use CSS Grid for doctor cards, HTML forms for booking, and JavaScript conditionals/DOM manipulation to pre-fill the form, validate required fields, and render the confirmation table.

Expected Output: A working booking flow: browse doctors, select one, fill and validate the form, and view an accurate confirmation summary.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a 'Find a Doctor' section with a grid of six doctor cards. Each card contains a circular profile picture placeholder, a name placeholder, and a specialty placeholder. Below the grid is a booking form with three input fields labeled 'DOCTOR', 'PREFERRED DATE', and 'PREFERRED TIME'. A 'BOOK APPOINTMENT' button is located at the bottom of the form.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots of doctor selection, form validation, and confirmation

Acceptance Criteria:

- ☐ Doctor cards are grouped correctly by department
- ☐ Selecting a doctor correctly pre-fills the booking form

- ☐ Form correctly blocks submission when required fields are missing
 - ☐ Confirmation summary accurately reflects the submitted booking
 - ☐ Logic correctly uses JavaScript conditionals and DOM manipulation
 - ☐ No major functional issues
-

Task 39 — Library Book Catalogue with Live Search-Style Filtering

Difficulty: Hard

Topics Covered:

- HTML Tables
- HTML Forms & Input Elements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Loops
- JS Conditional Statements

Objective: Build a library catalogue table that filters instantly as the user types into a search box, matching book title or author.

Task Description: A school library needs a digital catalogue page where staff can quickly find books by typing part of a title or author name into a search box, with matching rows highlighted instantly.

Functional Requirements:

- Display a table of at least 12 books with title, author, genre, and availability status
- Provide a text input that filters the table live as the user types
- Filtering matches against both title and author fields
- Show a 'No results found' message when nothing matches

Technical Requirements: Use an HTML table and a text input. Use JavaScript keyboard event listeners, loops to check every row, string comparison with conditionals, and DOM manipulation to show/hide rows.

Expected Output: A catalogue table that filters live and accurately as the librarian types, with a clear no-results state when nothing matches.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

Book Catalogue

SEARCH BY TITLE OR AUTHOR

SEARCH

TITLE	AUTHOR	GENRE	STATUS

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing an active search filter and a no-results state

Acceptance Criteria:

- ☐ At least 12 books are listed with complete details
- ☐ Typing in the search box filters rows live, without a submit button
- ☐ Search correctly matches partial title or author text
- ☐ No-results message appears correctly when nothing matches
- ☐ Filtering logic uses JavaScript loops, conditionals, and DOM manipulation
- ☐ No major functional issues

Task 40 — Event Management Landing Page with Speaker Grid & Registration

Difficulty: **Hard**

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Grid
- HTML Tables
- HTML Forms & Input Elements
- CSS Responsive Design & Media Queries
- JS Conditional Statements

Objective: Build a conference landing page with a speaker showcase, a schedule table, and a validated registration form.

Task Description: A tech conference needs a landing page that introduces the event, showcases speakers, lists the day's schedule, and lets attendees register through a validated form.

Functional Requirements:

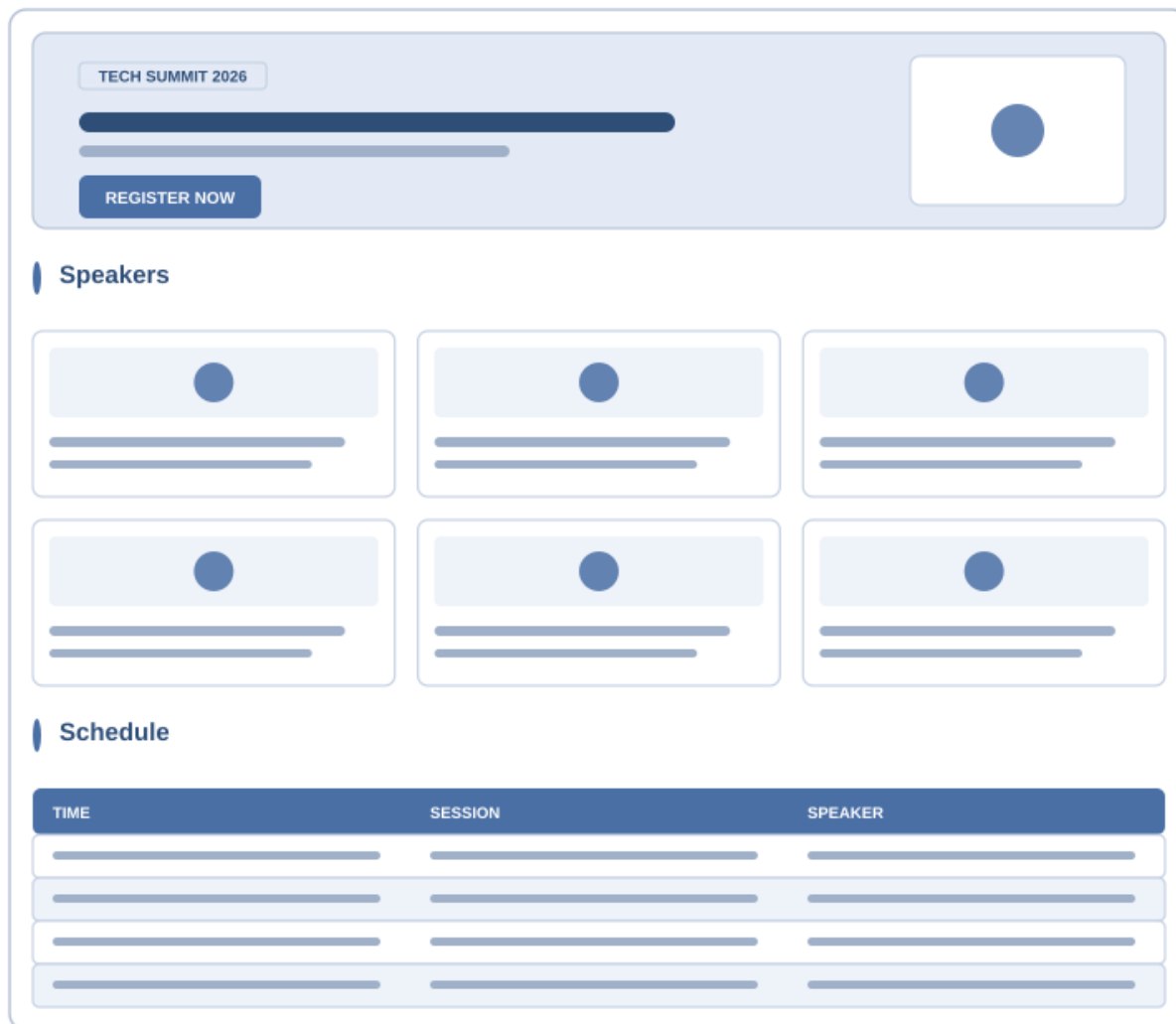
- Hero section introducing the event name, date, and location
- Speaker grid showing at least 6 speakers with photo placeholder, name, and topic
- Schedule table listing at least 6 sessions with time, title, and speaker
- Registration form validated with JavaScript before showing a confirmation message

Technical Requirements: Use CSS Grid for the speaker showcase, an HTML table for the schedule, HTML forms for registration, JavaScript conditionals/DOM manipulation for validation and confirmation, and media queries for responsiveness.

Expected Output: A complete, responsive conference landing page where all sections are populated correctly and registration is properly validated.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots of the full page and the registration confirmation

Acceptance Criteria:

- ☐ Hero, speaker grid, schedule table, and registration form are all present

- ☐ At least 6 speakers and 6 schedule sessions are shown with correct details
 - ☐ Registration form is validated and shows a confirmation on success
 - ☐ Page layout is responsive across at least two breakpoints
 - ☐ No major functional issues
-

Task 41 — Multi-Question Quiz Platform with Result Summary

Difficulty: **Hard**

Topics Covered:

- JS Variables, Data Types & Scope
- JS Functions & Objects
- JS Loops
- JS Conditional Statements
- JS Events (Mouse & Keyboard)
- CSS Display & Positioning
- CSS Responsive Design & Media Queries

Objective: Build a one-question-at-a-time quiz platform with progress indication, answer locking, and a detailed result summary screen.

Task Description: An assessment tool needs a more advanced quiz experience than a simple form: questions appear one at a time with a progress indicator, the selected answer locks in before moving on, and a final results screen breaks down correct vs. incorrect answers.

Functional Requirements:

- Show one question at a time from a bank of at least 8 questions
- Display a progress indicator (e.g., 'Question 3 of 8')
- Lock in the selected answer and disable changing it once 'Next' is clicked
- Final results screen shows total score and a per-question breakdown (correct/incorrect)

Technical Requirements: Store questions in JavaScript objects, use loops/conditionals to control quiz flow and scoring, event listeners for navigation and answer selection, and DOM manipulation/CSS display toggling to switch between question views and the results screen.

Expected Output: A polished, multi-step quiz experience ending in an accurate, detailed results breakdown.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The screenshot shows a quiz interface with a header 'Assessment' and 'Question 3 of 8'. Below this is a light blue box containing a text input field with the placeholder 'SELECT YOUR ANSWER' and a dark blue button labeled 'NEXT QUESTION'. At the bottom of the interface, there are three identical progress indicators, each consisting of a blue circle, a horizontal bar, and a shorter blue segment at the end of the bar.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots of a mid-quiz question and the final results screen

Acceptance Criteria:

- ☐ At least 8 questions are presented one at a time with a working progress indicator
- ☐ Answer selection is correctly locked once the user proceeds
- ☐ Final score and per-question breakdown are accurate
- ☐ Quiz flow correctly uses JavaScript objects, loops, and conditionals
- ☐ Interface is responsive and clearly usable
- ☐ No major functional issues

Task 42 — Banking Dashboard UI with Account Summary & Transfer Form

Difficulty: Hard

Topics Covered:

- CSS Grid
- CSS Flexbox
- HTML Tables
- HTML Forms & Input Elements
- JS Operators & Type Coercion
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a personal banking dashboard showing account balance cards, a transaction history table, and a validated funds-transfer form.

Task Description: A digital banking app needs a dashboard home screen where the user can see account balances at a glance, review recent transactions, and transfer funds between their own accounts with basic validation (e.g., cannot transfer more than the available balance).

Functional Requirements:

- Display at least 2 account balance cards (e.g., Savings, Checking)
- Transaction history table listing at least 6 recent transactions
- Transfer form to move funds between the displayed accounts
- Transfer is blocked with a clear message if the amount exceeds the source account's balance

Technical Requirements: Use CSS Grid/Flexbox for the dashboard layout, an HTML table for transactions, an HTML form for transfers, and JavaScript (variables, operators, conditionals, DOM manipulation, event listeners) to validate and process the transfer amount.

Expected Output: A dashboard where account balances, transaction history, and transfer validation all behave correctly and consistently.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a dashboard layout. At the top, there are two account balance cards, each with a circular profile icon and a horizontal bar representing the balance. Below these is a section titled 'Recent Transactions' with a table containing 6 rows of transaction data. The table has columns for DATE, DESCRIPTION, TYPE, and AMOUNT. At the bottom, there is a transfer form with two input fields for 'FROM ACCOUNT' and 'TO ACCOUNT', and a third input field for 'AMOUNT'. A blue 'TRANSFER' button is located below the 'AMOUNT' field.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing a successful transfer and a blocked over-limit transfer

Acceptance Criteria:

- ☐ Account balance cards and transaction table are present with correct data
- ☐ Transfer form correctly moves the entered amount between displayed balances
- ☐ Transfers exceeding available balance are correctly blocked with a message
- ☐ Balance and transaction updates are reflected immediately via DOM manipulation
- ☐ Logic correctly uses JavaScript operators and conditionals
- ☐ No major functional issues

Task 43 — Taxi/Cab Booking Interface with Fare Calculator

Difficulty: Hard

Topics Covered:

- HTML Forms & Input Elements
- CSS Flexbox
- JS Operators & Type Coercion
- JS Functions & Objects
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a cab booking interface where selecting a vehicle type and entering trip distance calculates an estimated fare.

Task Description: A ride-booking service needs a booking screen where the customer selects a vehicle type (e.g., Mini, Sedan, SUV) and enters an estimated distance, with the fare calculated instantly using a per-vehicle base fare and per-kilometre rate.

Functional Requirements:

- Display at least 3 vehicle type options with base fare and per-km rate
- Provide a distance input field
- Calculate and display the estimated fare instantly based on selected vehicle and distance
- Show a clear error message if distance is left empty or entered as a negative number

Technical Requirements: Use Flexbox for vehicle option cards, and JavaScript functions, arithmetic operators, conditionals, DOM manipulation, and event listeners to compute and validate the fare estimate.

Expected Output: A booking screen that reliably computes an accurate fare estimate for any valid combination of vehicle type and distance, and handles invalid input gracefully.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a 'Book a Ride' interface. At the top, there's a title 'Book a Ride' with a vertical bar icon. Below it, there are three vehicle options, each represented by a card with a blue circle icon, a horizontal bar, and the text '\$1.20/km'. Below these options is a section for input fields. It starts with a label 'PICKUP LOCATION' followed by a text input field. Below that is a label 'DISTANCE (KM)' followed by another text input field. At the bottom of this section is a blue button labeled 'GET FARE ESTIMATE'.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing a calculated fare and an error state

Acceptance Criteria:

- ☐ At least 3 vehicle types are shown with correct rate details
 - ☐ Fare calculation is accurate for the selected vehicle and entered distance
 - ☐ Empty or negative distance input is correctly blocked with an error message
 - ☐ Calculation logic correctly uses JavaScript functions and operators
 - ☐ No major functional issues
-

Task 44 — Online Exam Interface with Question Navigation Tracking

Difficulty: Hard

Topics Covered:

- HTML Forms & Input Elements
- CSS Grid
- JS Variables, Data Types & Scope
- JS Functions & Objects
- JS Loops
- JS Conditional Statements
- JS Events (Mouse & Keyboard)
- JS DOM Selection & Manipulation

Objective: Build an online exam interface with a question navigator panel that tracks answered, unanswered, and currently-viewed questions.

Task Description: An online examination system needs an interface showing the active question plus a grid of question numbers on the side, where each number's color indicates whether it has been answered, skipped, or is currently open, culminating in a final submission summary.

Functional Requirements:

- Display one question at a time from a bank of at least 10 questions
- Side panel shows a numbered grid for all questions, color-coded by status (answered / unanswered / current)
- Clicking any question number jumps directly to that question
- On final submit, show a summary of total answered vs. unanswered questions before confirming

Technical Requirements: Use CSS Grid for the question navigator panel. Use JavaScript objects/arrays to track question state, loops to render the navigator, conditionals to determine status colors, and event listeners/DOM manipulation to handle navigation and submission.

Expected Output: A working exam interface where the navigator panel accurately reflects question status at all times and direct navigation works correctly from any question number.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

Exam in Progress

Question 4

YOUR ANSWER

SAVE & NEXT

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing the navigator mid-exam and the final submission summary

Acceptance Criteria:

- ☐ At least 10 questions are supported with correct one-at-a-time display
- ☐ Navigator grid accurately reflects answered/unanswered/current status at all times
- ☐ Clicking a question number correctly jumps to that question
- ☐ Final summary accurately counts answered vs. unanswered questions
- ☐ Logic correctly uses JavaScript loops, conditionals, and DOM manipulation
- ☐ No major functional issues

Task 45 — Travel Agency Multi-Page Site with Package Cards & Enquiry Form

Difficulty: Hard

Topics Covered:

- HTML Structure & Semantic Elements
- HTML Links, Navigation & Media
- CSS Grid
- CSS Responsive Design & Media Queries
- HTML Forms & Input Elements
- JS Conditional Statements

Objective: Build a two-page travel agency site: a packages listing page and a booking-enquiry page, with validated form submission.

Task Description: A travel agency wants a packages page showcasing holiday packages as cards, and a separate enquiry page where interested customers submit their travel preferences, validated before showing a confirmation.

Functional Requirements:

- Packages page displays at least 6 travel package cards (destination, duration, price)
- Each package links to (or pre-fills) the enquiry page with that package's name

- Enquiry page form collects name, email, travel dates, and number of travellers
- Form is validated with JavaScript, showing a confirmation message on success

Technical Requirements: Use CSS Grid for the package listings, HTML links between the two pages, and JavaScript conditionals/DOM manipulation for form validation and confirmation.

Expected Output: A working two-page flow from browsing packages to submitting a validated, package-specific enquiry.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (both HTML pages, shared CSS/JS files)
- Screenshots of the packages page and a submitted enquiry

Acceptance Criteria:

- ☐ At least 6 travel packages are displayed with correct details
- ☐ Navigation from packages page to enquiry page works correctly
- ☐ Enquiry form correctly validates required fields before submission
- ☐ Confirmation message appears correctly on valid submission
- ☐ Pages share consistent styling and structure
- ☐ No major functional issues

Task 46 — Music Playlist Interface with Play/Pause State & Track Highlighting

Difficulty: Hard

Topics Covered:

- CSS Flexbox
- CSS Pseudo-classes & Pseudo-elements
- JS Variables, Data Types & Scope

- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Conditional Statements

Objective: Build a static music player interface where clicking a track highlights it as active and toggles a play/pause button state.

Task Description: A music streaming demo needs a player interface with a track list and a control bar; clicking a track marks it as the active/playing track, and a play/pause button visually toggles between its two states.

Functional Requirements:

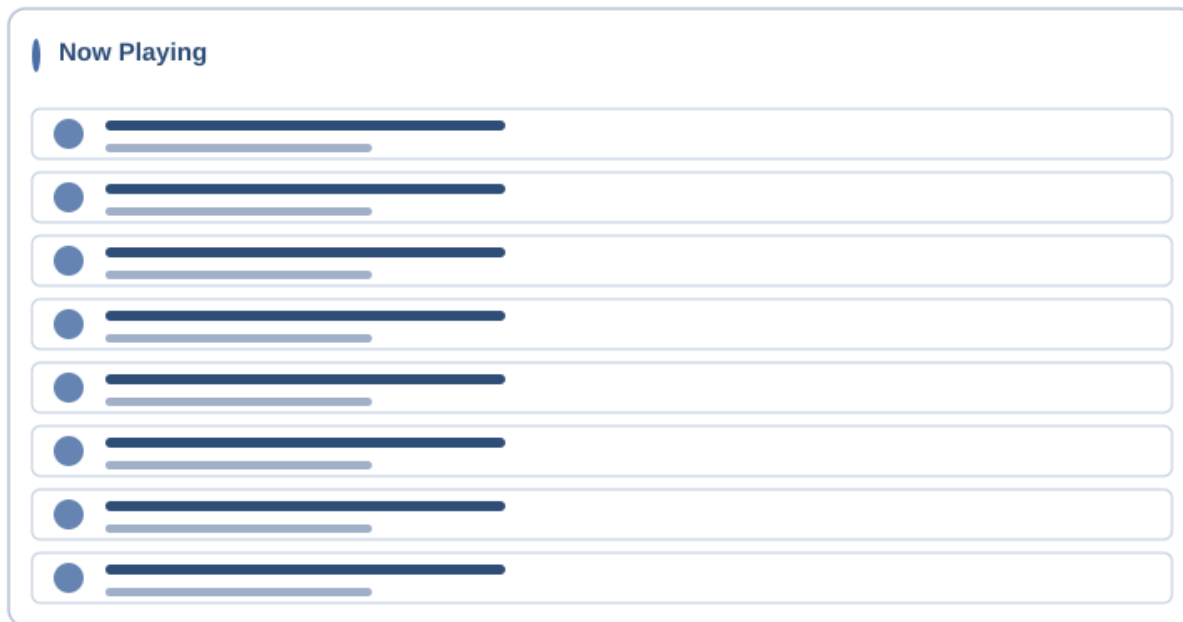
- Display a playlist of at least 8 tracks with title and artist
- Clicking a track highlights it as the 'now playing' track (only one at a time)
- A play/pause button toggles its icon/label and state on click
- A bottom control bar displays the currently selected track's title and artist

Technical Requirements: Use Flexbox for the player layout, pseudo-classes for hover/active track styling, and JavaScript (variables, conditionals, event listeners, DOM manipulation) to manage the active track and play/pause state.

Expected Output: A player interface where track selection and play/pause state are always visually accurate and only one track is highlighted as active at a time.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing a selected track and both play/pause states

Acceptance Criteria:

- ☐ At least 8 tracks are listed with correct details
- ☐ Selecting a track correctly highlights it and updates the control bar
- ☐ Only one track is marked active at any time
- ☐ Play/pause button correctly toggles between two distinct states

- ☐ Logic correctly uses JavaScript variables, conditionals, and DOM manipulation
- ☐ No major functional issues

Task 47 — Social Media Profile Page with Tab Switching

Difficulty: Hard

Topics Covered:

- HTML Structure & Semantic Elements
- CSS Flexbox
- CSS Grid
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- CSS Display & Positioning

Objective: Build a social media profile page with Posts, About, and Photos tabs that switch content without reloading the page.

Task Description: A social networking demo needs a user profile page with a cover banner, profile info, and three tabs (Posts, About, Photos) where clicking a tab shows only the relevant content panel.

Functional Requirements:

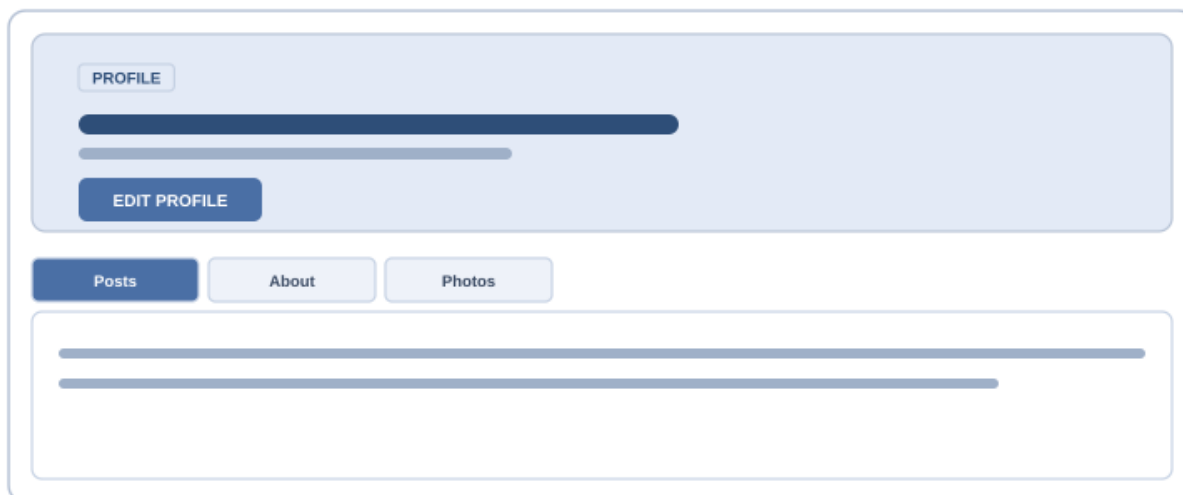
- Profile header with cover banner, avatar placeholder, name, and bio
- Three tabs: Posts (grid/list of post placeholders), About (bio details), Photos (image grid)
- Clicking a tab shows only that tab's content and visually marks the tab as active
- Only one tab's content is visible at any time

Technical Requirements: Use Flexbox/Grid for the profile layout, and JavaScript event listeners, DOM manipulation, and CSS display toggling to control which tab panel is visible.

Expected Output: A profile page where tab switching is instant, reliable, and always shows exactly one content panel at a time.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshots showing each of the three tabs active

Acceptance Criteria:

- ☐ Profile header and all three tabs are present and correctly structured
 - ☐ Clicking each tab correctly shows only its own content
 - ☐ Active tab is visually indicated at all times
 - ☐ No more than one tab panel is visible at once
 - ☐ Logic correctly uses JavaScript events and DOM manipulation
 - ☐ No major functional issues
-

Task 48 — Recipe Sharing Website with Category Filter & Detail Page

Difficulty: **Hard**

Topics Covered:

- HTML Structure & Semantic Elements
- HTML Links, Navigation & Media
- CSS Grid
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)
- JS Loops
- JS Conditional Statements

Objective: Build a two-page recipe site: a filterable recipe listing page and a recipe detail page with ingredients and steps.

Task Description: A recipe-sharing hobby site needs a listing page where visitors filter recipes by category (e.g., Breakfast, Dinner, Dessert) using JavaScript, and a separate detail page template showing full ingredients and preparation steps for one recipe.

Functional Requirements:

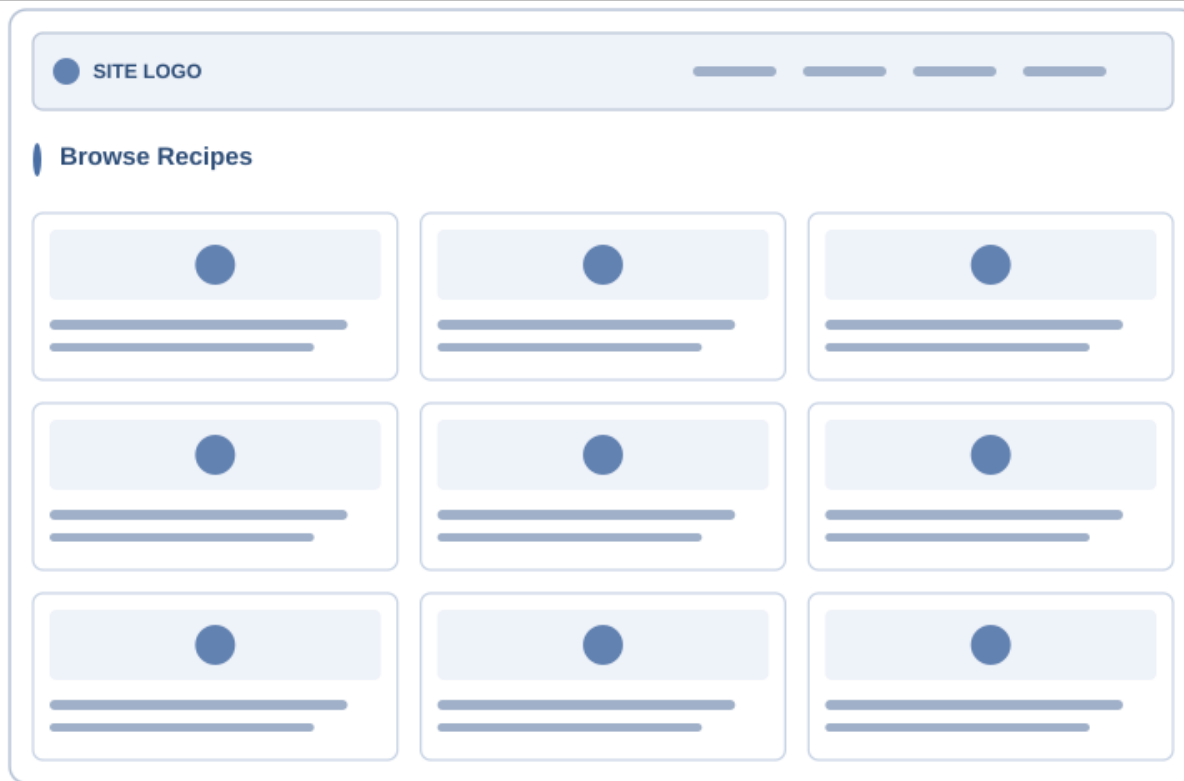
- Listing page shows at least 9 recipe cards tagged with a category
- Category filter buttons show only matching recipes using JavaScript (no reload)
- Each recipe card links to a detail page showing ingredients (list) and steps (ordered list)
- Both pages share consistent header, navigation, and footer styling

Technical Requirements: Use CSS Grid for the recipe listing. Use JavaScript loops and conditionals for category filtering with DOM manipulation. Use HTML lists (ul/ol) on the detail page for ingredients and steps.

Expected Output: A working recipe browsing experience: filter recipes by category on the listing page, then view full details including ingredients and steps on the detail page.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (both HTML pages, shared CSS/JS files)
- Screenshots of a filtered listing and a recipe detail page

Acceptance Criteria:

- ☐ At least 9 recipes are shown with correct category tagging
- ☐ Category filtering works correctly via JavaScript without reloading
- ☐ Detail page correctly displays ingredients as a list and steps as an ordered list
- ☐ Both pages share consistent styling and navigation
- ☐ No major functional issues

Task 49 — Car Rental Booking Interface with Vehicle Selection & Price Summary

Difficulty: Hard

Topics Covered:

- CSS Grid
- CSS Flexbox
- HTML Forms & Input Elements
- JS Operators & Type Coercion
- JS Functions & Objects
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a car rental booking interface where selecting a vehicle and rental duration calculates an accurate price summary, including optional add-ons.

Task Description: A car rental company needs a booking page where customers pick a vehicle card, enter rental duration (days), optionally add extras (e.g., GPS, insurance), and see a live, itemized price summary before booking.

Functional Requirements:

- Display at least 4 vehicle cards with daily rate and specifications
- Provide a rental duration input (number of days)
- Provide at least 2 optional add-ons with checkboxes and their own daily cost
- Live price summary breaks down vehicle cost, add-on cost, and grand total

Technical Requirements: Use CSS Grid/Flexbox for vehicle cards, HTML form elements for duration/add-ons, and JavaScript (functions, arithmetic operators, conditionals, event listeners, DOM manipulation) to compute and display the itemized summary live.

Expected Output: A booking interface where any combination of vehicle, duration, and add-ons produces an accurate, itemized live price summary.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.

The mockup shows a light blue rounded rectangle containing the booking interface. At the top left is a section header 'Select a Vehicle' with a vertical bar icon. Below this are four vehicle cards arranged horizontally. Each card has a light blue header with a dark blue circle, a light blue body with two horizontal bars, and a light blue footer with the text '\$45/day'. Below the vehicle cards is a section with two input fields: 'RENTAL DAYS' and 'ADD-ONS'. Below these fields is a dark blue button with the text 'VIEW SUMMARY'.

Deliverables:

- Source code (HTML, CSS, JS files)
- Screenshot showing a full price breakdown with add-ons selected

Acceptance Criteria:

- ☐ At least 4 vehicles are shown with correct daily rates
- ☐ Duration input correctly affects the calculated totals
- ☐ Add-on checkboxes correctly add/remove their cost from the summary
- ☐ Itemized summary (vehicle cost, add-ons, total) is accurate at all times
- ☐ Logic correctly uses JavaScript functions and operators
- ☐ No major functional issues

Task 50 — Multi-Page Portfolio & Blog Website with Theme Toggle

Difficulty: Hard

Topics Covered:

- HTML Structure & Semantic Elements
- HTML Links, Navigation & Media
- CSS Grid
- CSS Flexbox
- CSS Responsive Design & Media Queries
- CSS Cascade, Specificity & Inheritance
- CSS Math Functions (calc/min/max/clamp)
- HTML Forms & Input Elements
- JS Conditional Statements
- JS DOM Selection & Manipulation
- JS Events (Mouse & Keyboard)

Objective: Build a complete multi-page personal portfolio and blog website with a validated contact form and a working light/dark theme toggle.

Task Description: A developer wants a full personal site: a portfolio home page, a blog listing page, and a contact page, all sharing a consistent design system, with a theme toggle button that switches the entire site between light and dark appearance.

Functional Requirements:

- Home page with hero, skills, and featured-projects sections
- Blog page listing at least 6 article cards in a responsive grid
- Contact page with a validated contact form (name, email, message)
- A theme toggle button (visible on every page) switches all pages between light and dark styling
- Selected theme styling is visually consistent across all three pages

Technical Requirements: Use CSS Grid/Flexbox across all pages with a shared stylesheet (demonstrating cascade), the clamp()/calc() math functions for fluid heading sizes and spacing, media queries for responsiveness, and JavaScript DOM manipulation (toggling a class on the page) plus event listeners to implement the theme toggle and form validation.

Expected Output: A cohesive three-page personal website where navigation, responsiveness, contact validation, and the light/dark theme toggle all work correctly and consistently.

Reference UI / Expected Output Image:

Original layout mockup — not a screenshot; for structural reference only.



Deliverables:

- Source code (all HTML pages, shared CSS/JS files)
- Screenshots of all three pages in both light and dark themes

Acceptance Criteria:

- ☐ All three pages (Home, Blog, Contact) are present, linked, and consistently styled
- ☐ Theme toggle correctly switches styling site-wide and persists across the visible session
- ☐ Contact form is correctly validated before showing a confirmation
- ☐ Layout is responsive across at least two breakpoints on every page
- ☐ Blog grid displays at least 6 articles correctly
- ☐ No major functional or navigation issues